Instruction-Level Parallelism

Topics:

- Pipelining
- Superscalar processors
- VLIW architecture

Overview

Modern processors apply techniques for executing several instructions in parallel to enhance the computing power.

The potential of executing machine instructions in parallel is called instruction level parallelism (ILP).

Remember: execution of one instruction is broken into several steps

Pipelining:

Different steps of multiple instructions are executed simultaneously.

Concurrent execution:

The same steps of multiple machine instructions may be executed simultaneously.

Requires multiple functional units

Techniques: Superscalar
VLIW (very long instruction word)

Pipelining: principle

Principle:

The execution of a machine instruction is divided into several steps – called *pipeline stages* - taking nearly the same execution time. These stages may be executed in parallel.

Example MIPS: 5 pipeline stages

1. IF: instruction fetch
2. ID: instruction decode and register file read
3. EX: execution / memory address calculation
4. MEM: data memory access
5. WB: result write back

Pipelining: principle

Executing two 5-step instructions (e.g. `lw`) without pipelining

Instruction 1:	S1	S2	S3	S4	S5					
Instruction 2:						S1	S2	S3	S4	S5
Clock cycle:	1	2	3	4	5	6	7	8	9	10

Executing 6 instructions using pipelining

Instruction 1:	S1	S2	S3	S4	S5					
Instruction 2:		S1	S2	S3	S4	S5				
Instruction 3:			S1	S2	S3	S4	S5			
Instruction 4:				S1	S2	S3	S4	S5		
Instruction 5:					S1	S2	S3	S4	S5	
Instruction 6:						S1	S2	S3	S4	S5
Clock cycle:	1	2	3	4	5	6	7	8	9	10

Pipelined MIPS datapath

In this chapter we will design a pipelined MIPS datapath for the following instructions: `lw, sw, add, sub, and, or, slt, beq`

Situations may occur where two instructions cannot be executed in the pipeline right after each other!

Example: Non-pipelined multi-cycle CPU has shared ALU for

1. executing arithmetic/logical instructions
2. incrementing PC

Structural hazard: Two instructions wish to use a certain hardware component in the same clock cycle leading to a *resource conflict*.

For RISC instruction sets structural hazards can often be resolved by additional hardware.

Pipelined MIPS datapath

Additional Hardware being required:

1. Permit incrementing PC and executing arithmetic/logical instructions concurrently:

   use separate adder for incrementing PC

2. Permit reading next instruction and reading/writing data from/to memory:

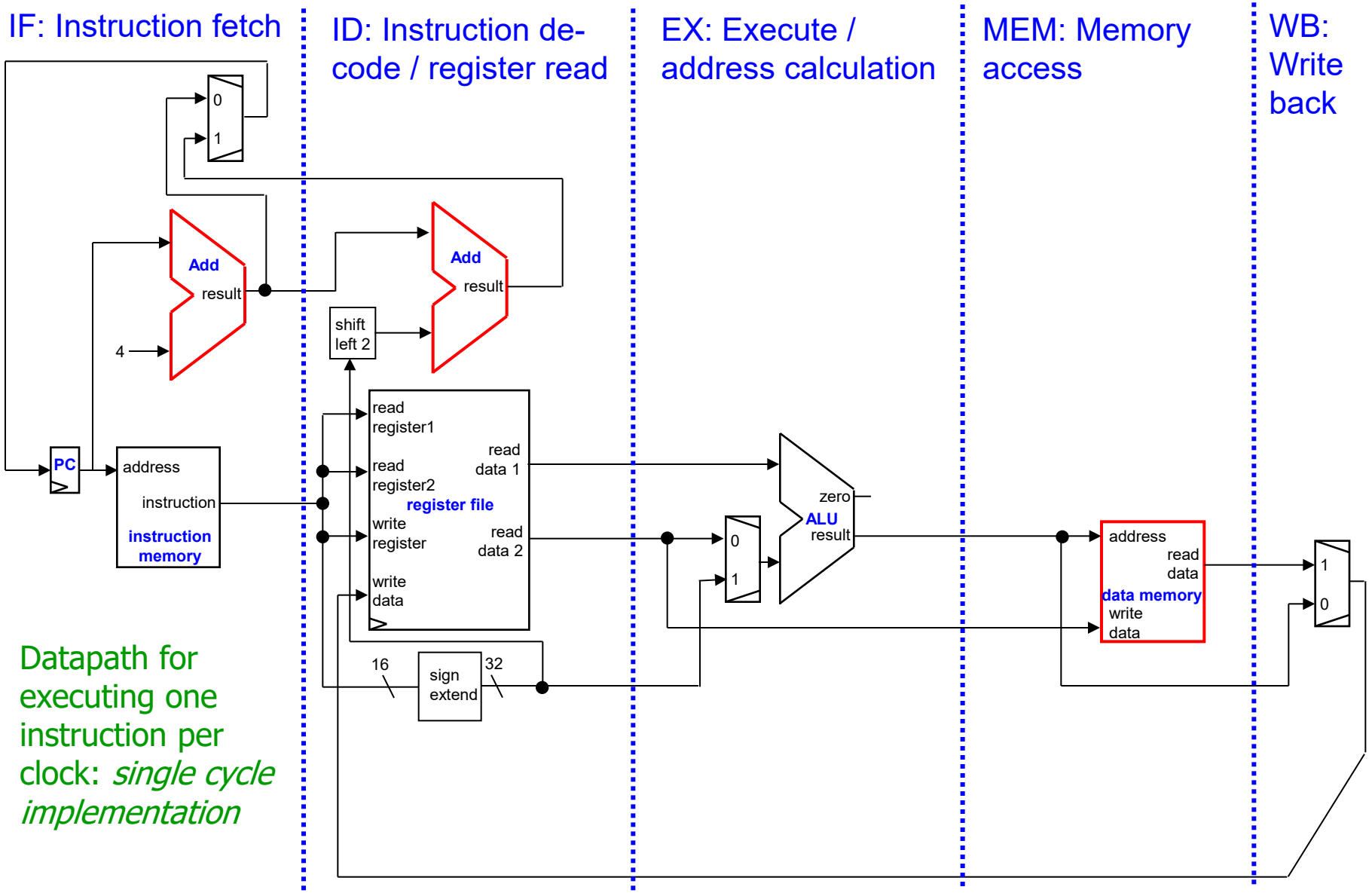
   divide memory into instruction memory and data memory
   (*Harvard architecture*)

3. Permit executing an arithmetic/logical instruction (uses ALU in 3. cycle) followed by a branch (calculates branch target in 2. cycle):

   use separate adder for branch address calculation

Duplicating hardware components in general leads to less and/or smaller multiplexers.

MIPS datapath (without pipelining)



Datapath for executing one instruction per clock: *single cycle implementation*

Pipelined MIPS datapath

Pipelined MIPS datapath additionally requires

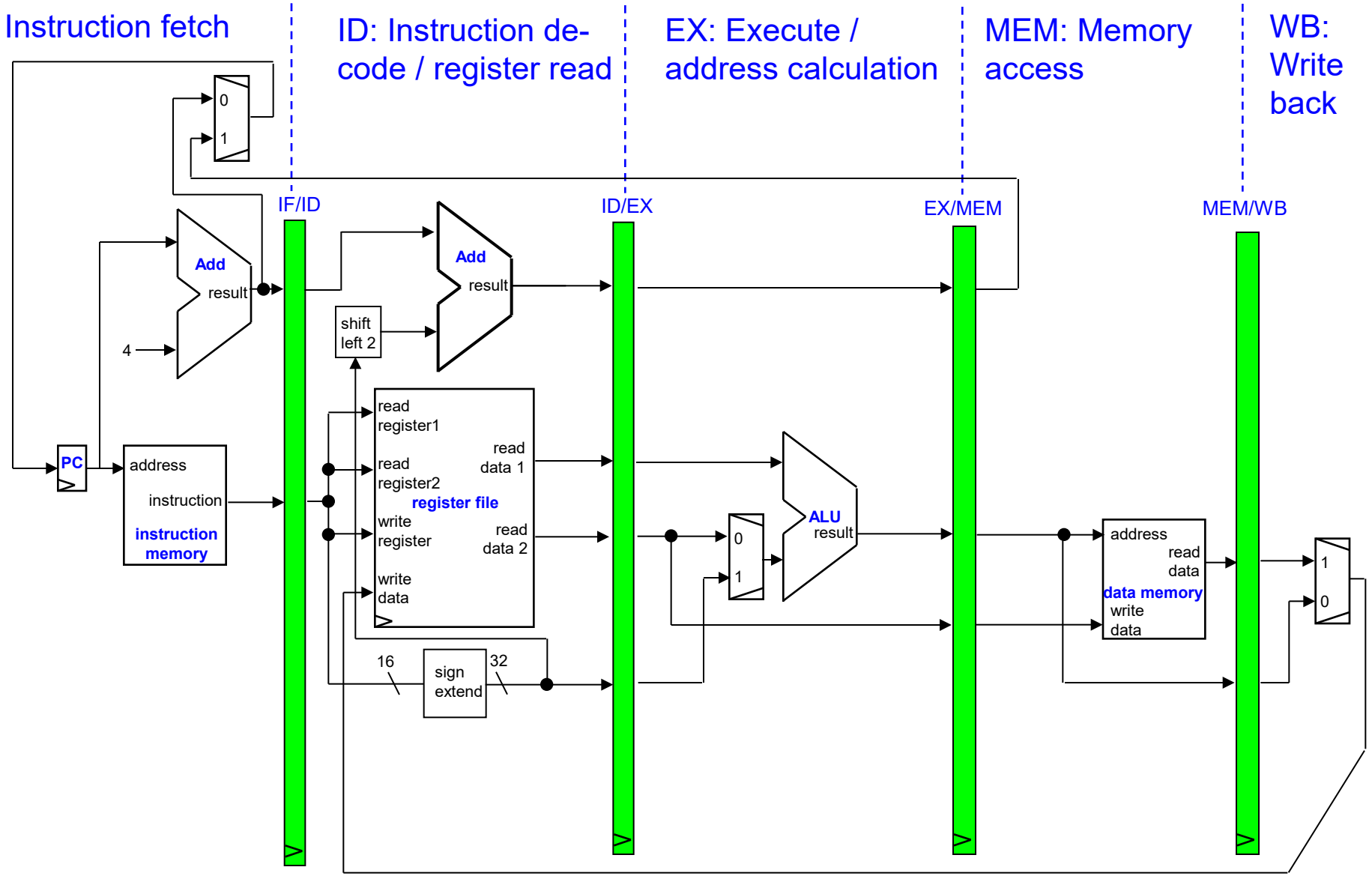
Pipeline registers:

- Store all the data occurring at the end of one pipeline stage that are required as input data in the next stage
- Divide datapath into pipeline stages
- Replace temporary datapath registers of non-pipelined multi-cycle implementation, e.g.:

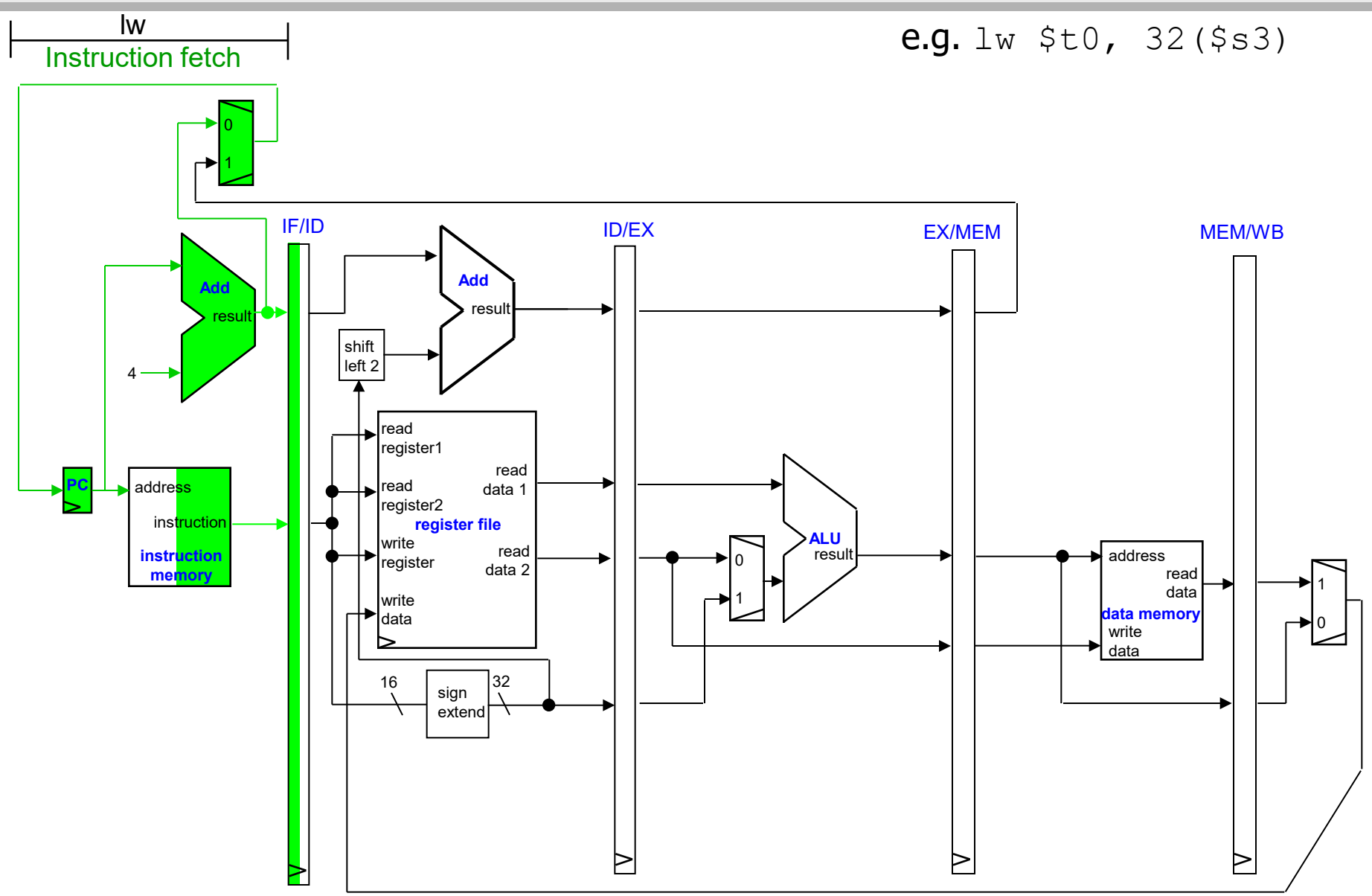
  ALU target register T replaced by pipeline register EX/MEM

  Instruction register IR replaced by pipeline register IF/ID

Pipelined MIPS datapath

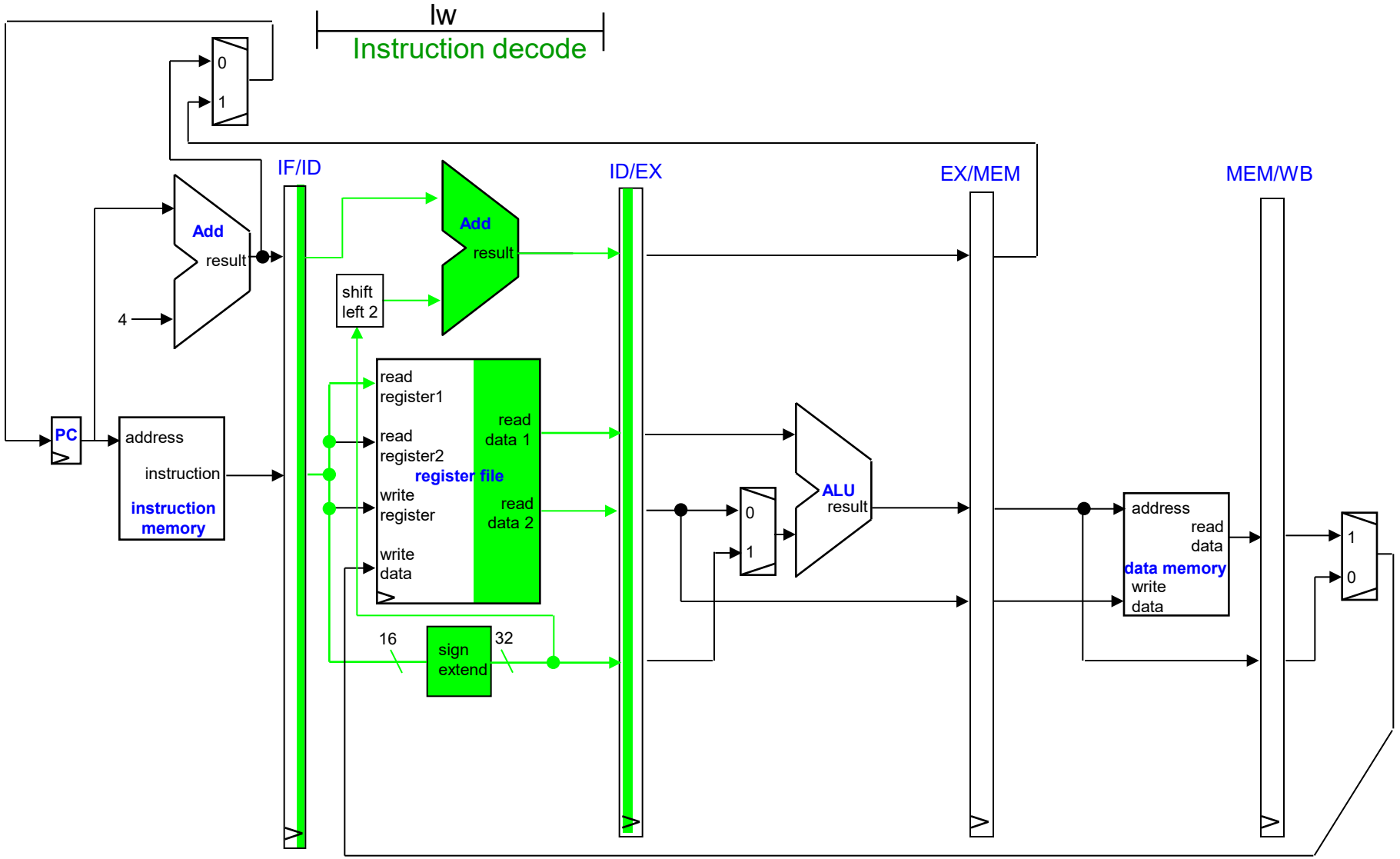


Executing an instruction, phase 1: instruction fetch



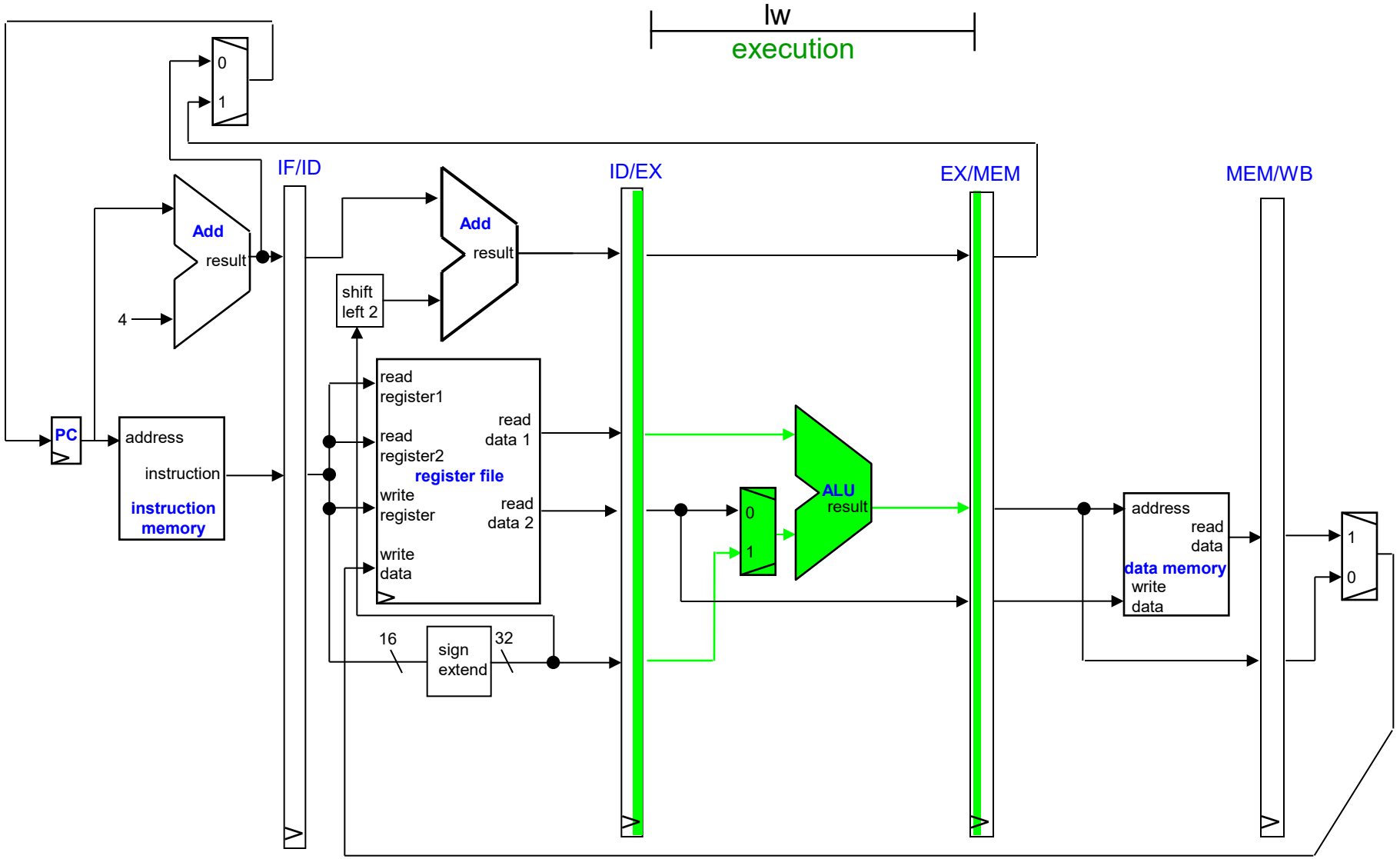
Executing an instruction, phase 2: instruction decode

e.g. `lw $t0, 32($s3)`

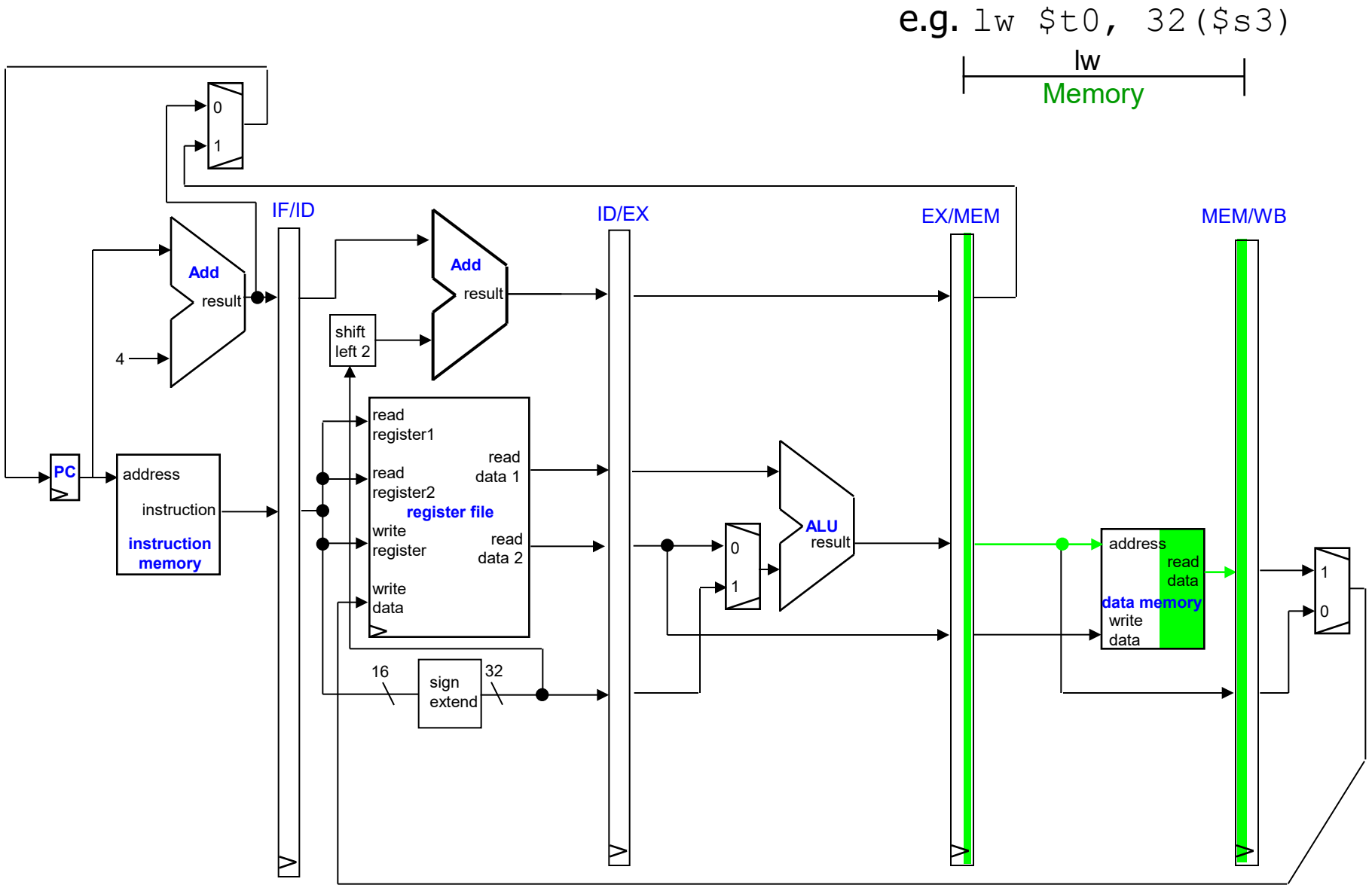


Executing an instruction, phase 3: execution

e.g. `lw $t0, 32($s3)`

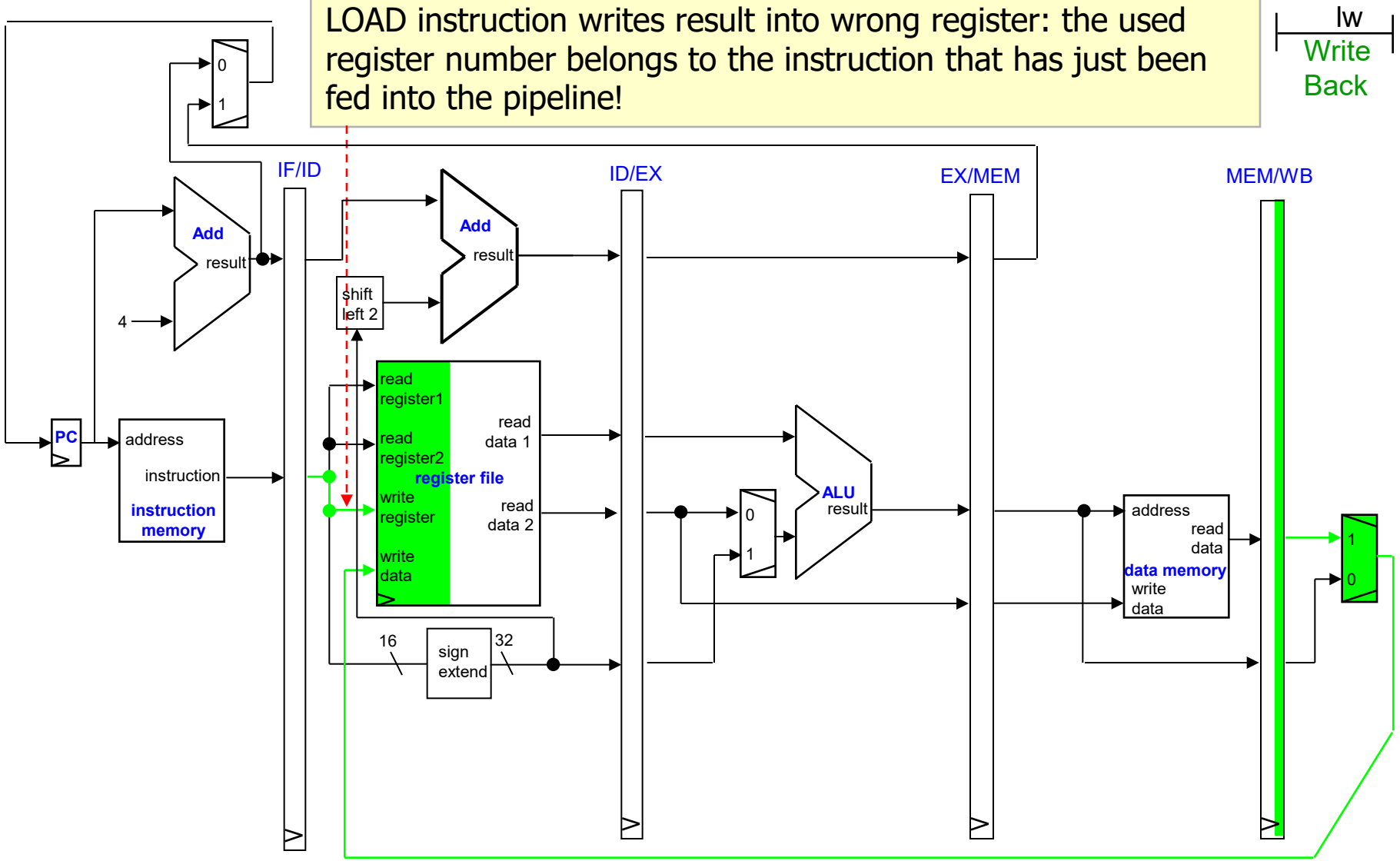


Executing an instruction, phase 4: memory access



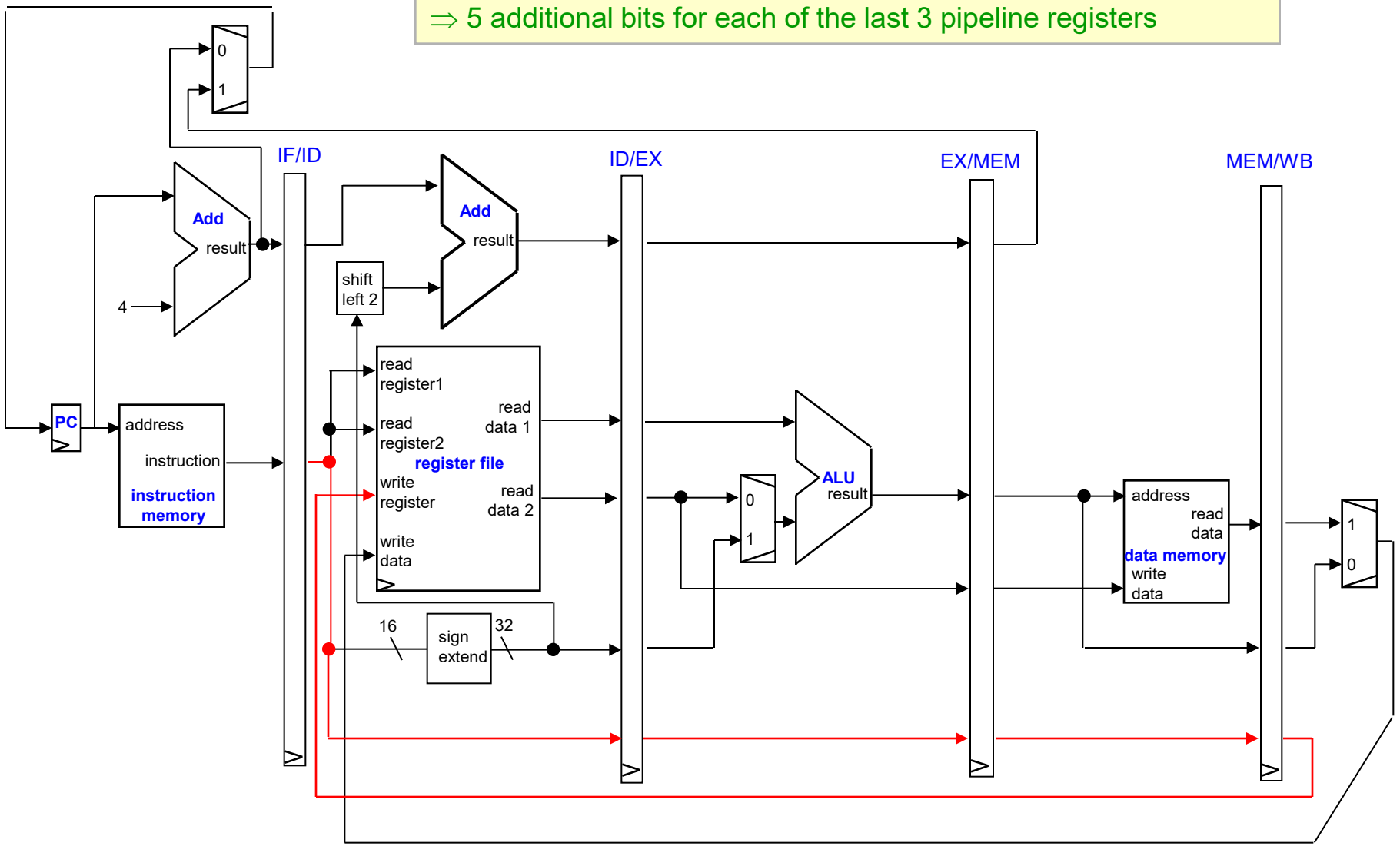
Executing an instruction, phase 5: write back

BUG !!
LOAD instruction writes result into wrong register: the used register number belongs to the instruction that has just been fed into the pipeline!



Revised hardware

Solution: keep register number and pass it to the last stage
⇒ 5 additional bits for each of the last 3 pipeline registers



Control for pipelined MIPS processor

General Approach:

In stage ID, create all control signals which are needed for an instruction in subsequent stages (EX, MEM, WB) and store them in the ID/EX pipeline register.

Then, in each clock cycle hand over control signals to the next stage using the corresponding pipeline registers.

We can divide the control signals into 5 groups corresponding to the pipeline stages where they are needed.

Which signals are required in which stage?

Control for pipelined MIPS processor

1. Instruction fetch:
Instruction memory is read and PC is written in every clock cycle ⇒ no control signals required!

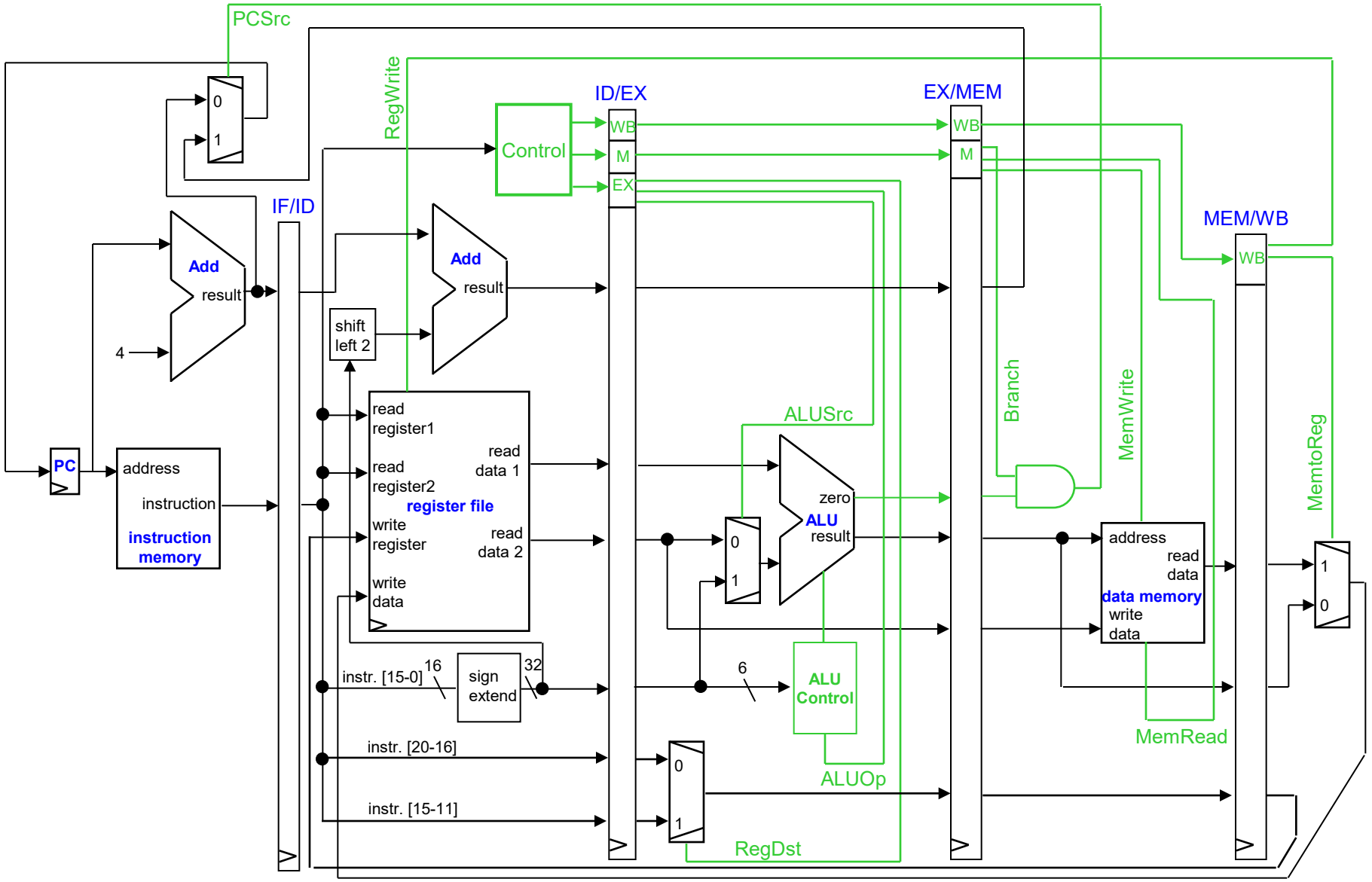
2. Instruction decode / register file read:
The same operations are performed in every clock cycle ⇒ no control signals required!

3. Execute / address calculation:
ALUop and *ALUsrc* (as described in Chapter 5), *RegDst* (use *rd* or *rt* as target)

4. Memory access:
MemRead and *MemWrite* (control data memory): set by `lw`, `sw`
Branch (PC will be reloaded if condition is fulfilled): set by `beq`
PCsrc is determined from *Branch* and *zero* (from ALU, condition is fulfilled if set)

5. Write back:
MemtoReg (send either ALU result or memory value to register file)
RegWrite (register file write enable)

Pipelined MIPS data path and control



Example

Consider the following program

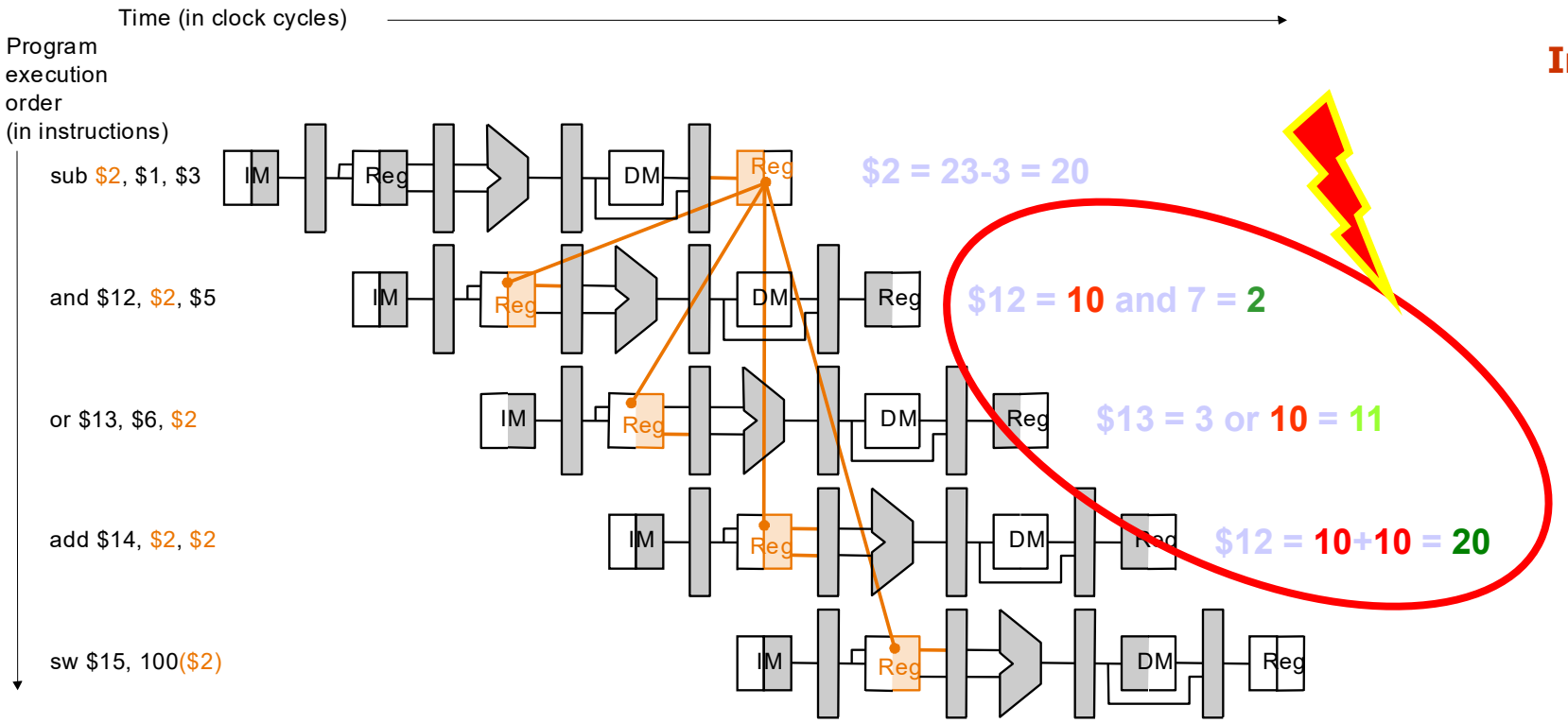
```
sub $2,  $1, $3       #  $2  = 23-3 = 20
and $12, $2, $5       #  $12 = 20 and 7 = 4
or  $13, $6, $2       #  $13 = 3 or 20 = 23
add $14, $2, $2       #  $14 = 20+20= 40
sw  $15, 100($2)      #  save $15 to 100(20)
```

Assume the following initial register contents:

$1 = 23
$2 = 10
$3 = 3
$5 = 7
$6 = 3

Data dependences and hazards

Consider in the following only data hazards for register-register-type instructions



Initial values:
$1 = 23
$2 = 10
$3 = 3
$5 = 7
$6 = 3

Data dependence leading to error (hazard)!

Dependences

Consider an instruction *a* that precedes an instruction *b* in program order:

A *data dependence* between *a* and *b* occurs when *a* writes into a register that will be read by *b.*

An *antidependence* between *a* and *b* occurs when *b* writes into a register that is read by *a*.

An *output dependence* between *a* and *b* occurs when both, *a* and *b* write into the same register.

A *data hazard* is created whenever the overlapping (pipelined) execution of *a* and *b* would change the order of access to the operands which are involved in the dependency.

Data hazards

Consider an instruction *a* that precedes an instruction *b* in program order:

Depending on the type of the dependence between *a* and *b* the following hazards may occur:

RAW: read after write

b reads a source before *a* writes it, so *b* incorrectly gets the old value.

WAR: write after read

b writes an operand before it is read by *a*, so *a* incorrectly gets the new value.

WAW: write after write

b writes an operand before it is written by *a*, leaving the wrong result in the target register.

In the following we consider only data hazards for R-type instructions

Software solution for resolving data hazards

Compiler resolves all data hazards:

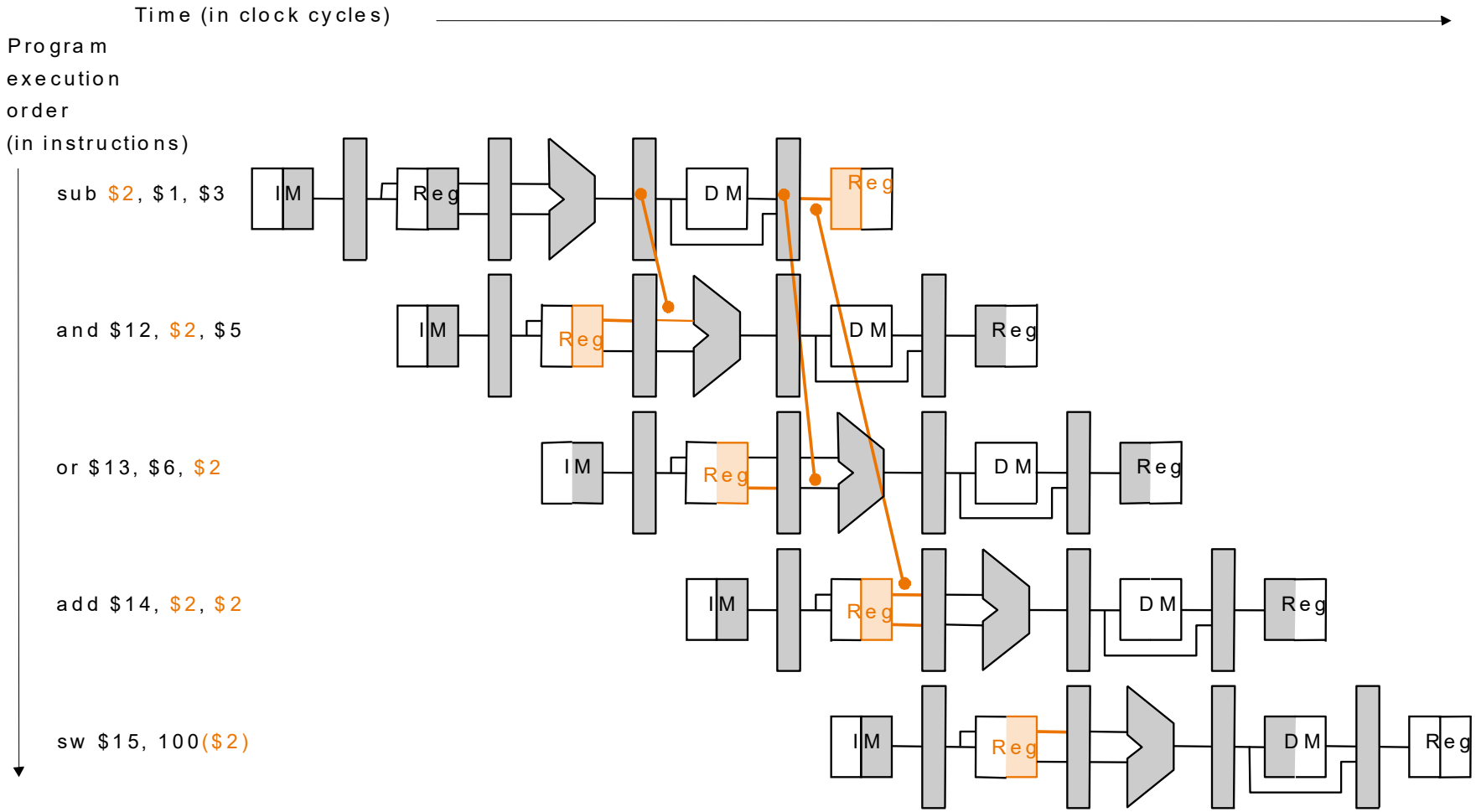
- Test machine language program for potential data hazards
- Eliminate them by inserting NOP – instructions (*no operation*)

Example:

```
sub $2, $1, $3
nop
nop
nop
and $12, $2, $5
or  $13, $6, $2
add $14, $2, $2
sw  $15, 100($2)
```

Modern processors are able to detect data hazards during program execution by analyzing the register numbers of the instructions using additional control logic!

Hardware solution for resolving data hazards



Data required by subsequent instructions exists already in pipeline register!

Register file: If a register is read and written in the same clock cycle, send new data to data output!

MIPS datapath using forwarding

Forwarding

ALU may read operands from each of the pipeline registers.

The correct operands are selected by multiplexers that are controlled by an additional control unit: *forwarding unit*

<u>Forwarding unit gets as input:</u>

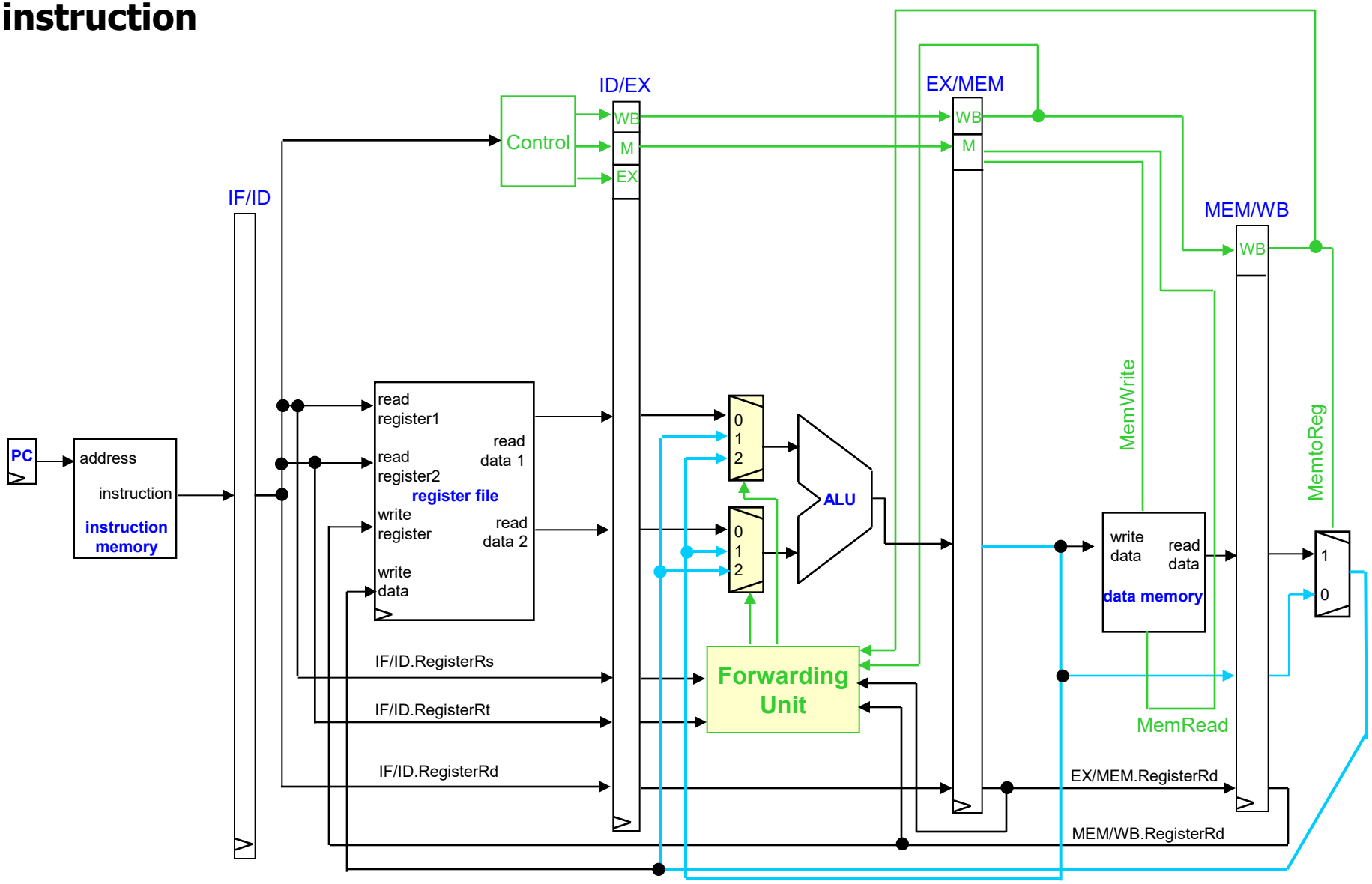
- Register operand numbers of instruction in EX stage
- Target register number of instructions being in MEM and WB stage
- Control signals indicating type of instructions being in MEM and WB stage

Register numbers are stored and moved forward in the pipeline registers

For reasons of clarity the hardware structure shown on the following slide has been simplified. Adder for branch target calculation, ALU input for address calculation and address input of data memory are missing.

MIPS datapath using forwarding

For R-type instruction



Forwarding

- Data hazards may be resolved if the operands being read by the instruction in the EX stage are already stored in one of the pipeline registers!

- Now consider the following program:

```
lw    $2, 20($1)
and   $4, $2, $5
or    $8, $2, $6
add   $9, $4, $2
slt   $1, $6, $7
```

AND instruction requires $2 at the beginning of the 3. stage (4. cycle)

BUT: value for $2 is stored in a pipeline register at the end of stage 4 of LW (4. cycle)

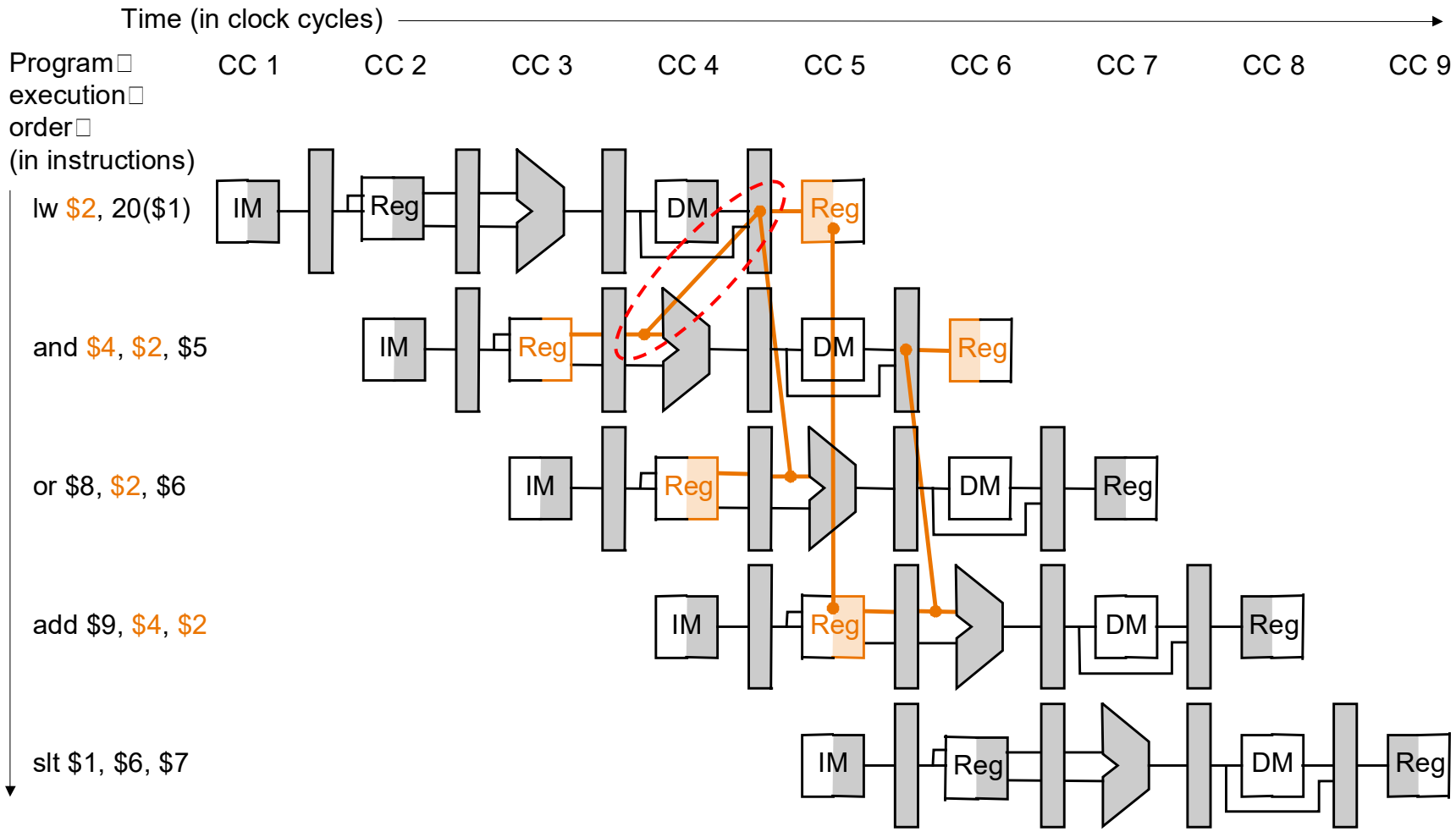
⇒ hazard may not be resolved by forwarding

We have to ***stall*** the pipeline for combinations of a *load* followed by an instruction that reads its result!

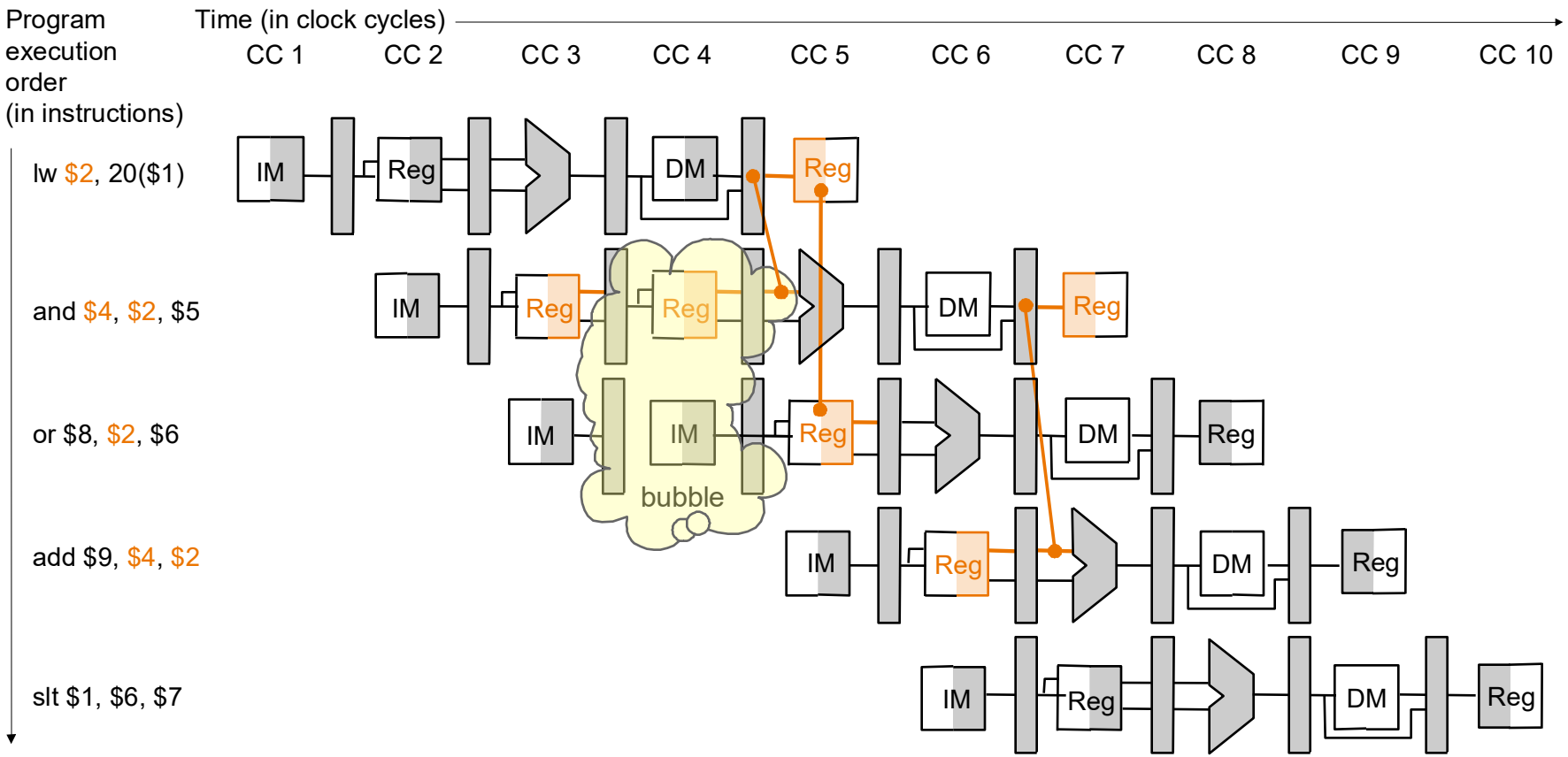
Additional hardware for detecting hazards and stalling the pipeline:

Hazard detection unit

Illustration



Stalling the pipeline



Stalling the pipeline means to repeat all actions from the previous clock cycle in the corresponding stages.

PC and IF/ID register must be prevented from being overwritten.

Control Hazards

Consider the following program:

```
        beq    $1,  $3, L0   # PC relative addressing
        and    $12, $2, $5
        or     $13, $6, $2
        add    $14, $2, $2
        ...
L0:     lw     $4, 50($14)
```

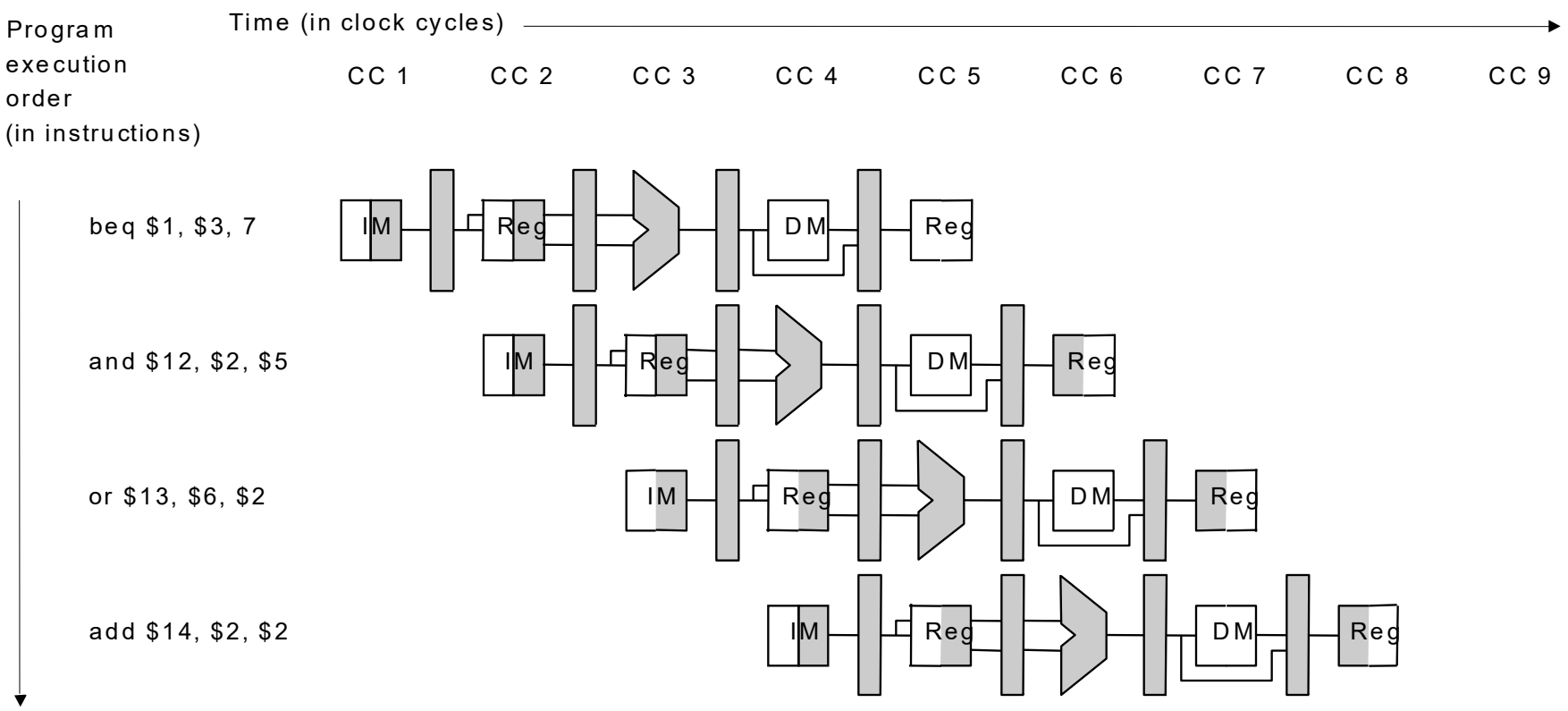
Efficient pipelining: one instruction is fetched at every clock cycle

BUT: Which instruction has to be executed after the branch?

"Control hazard" (or "branch hazard"):
We start executing instructions before we know whether they are really part of the program flow!

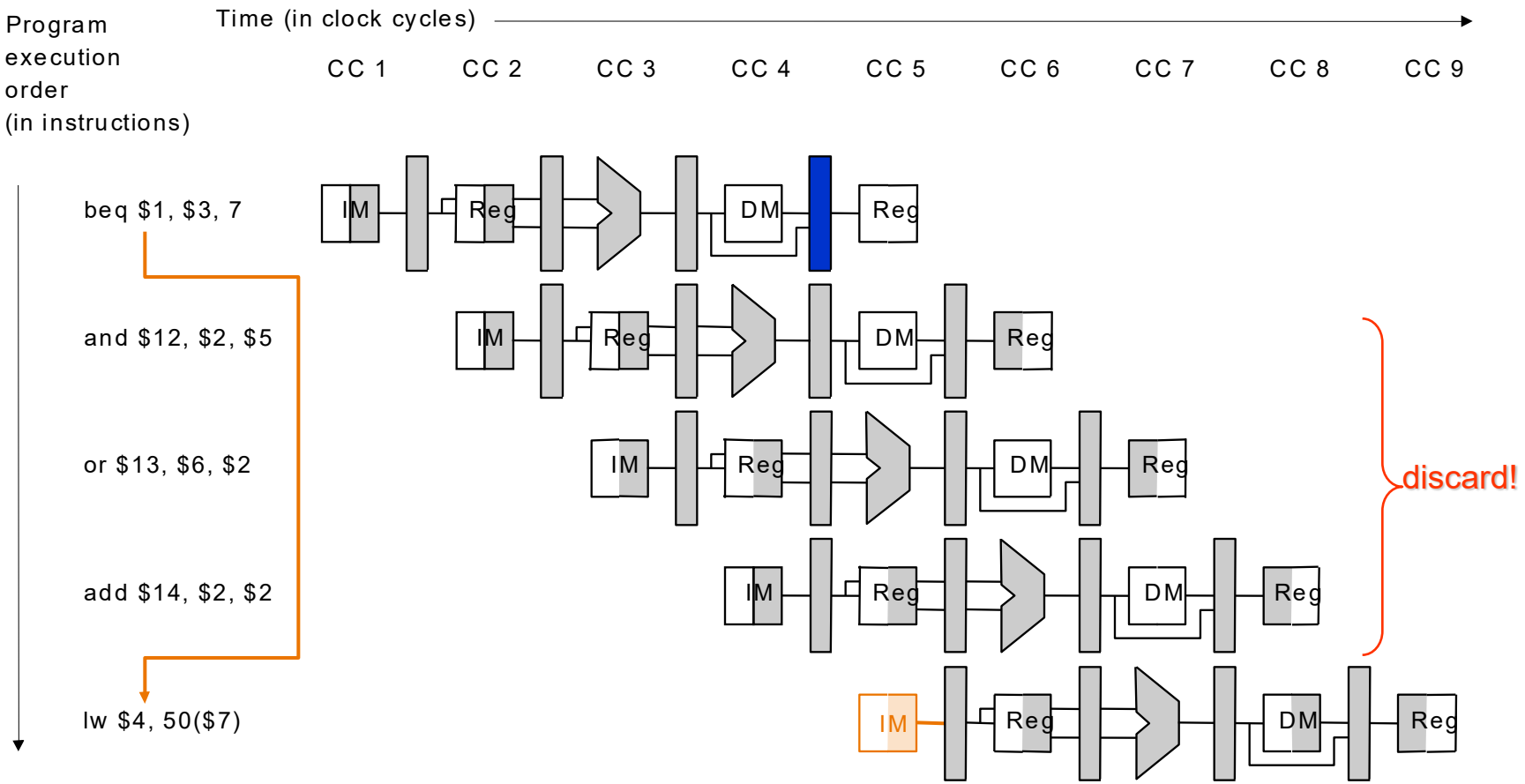
Strategy: Assume branch not taken

For every branch we assume that it is not taken and we begin executing the subsequent instructions $pc+4, $pc+8 and $pc+12.



Assume Branch not taken (continued)

However, if the branch is taken we have to discard all instructions from the pipeline!



In CC5 all data calculated in the stages ID, EX and MEM have to be marked as invalid!

⇒ Set control signals for writing of memory and register file to zero

Reducing the delay of branches

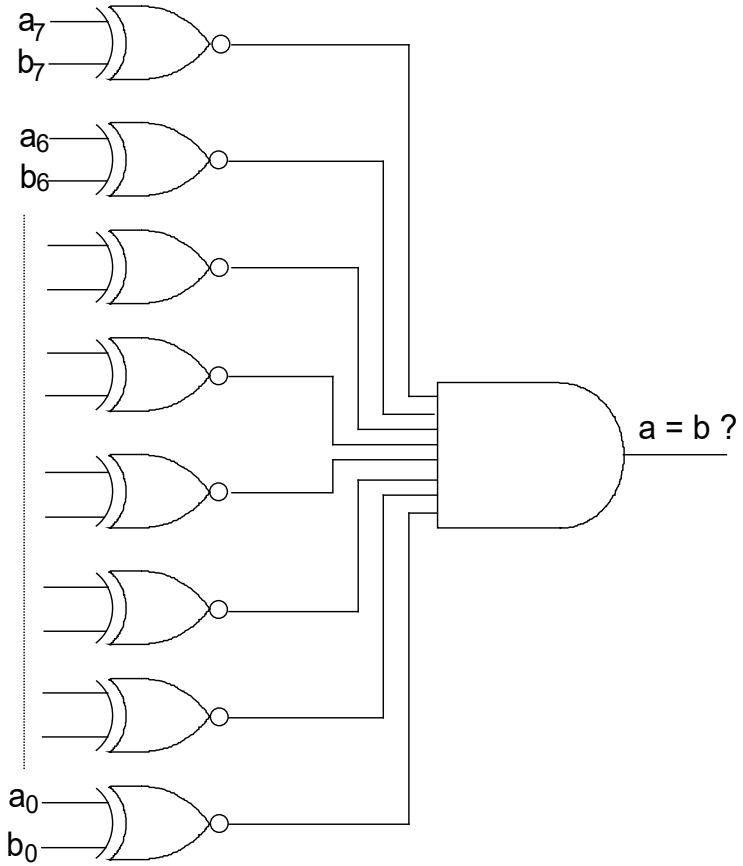
The earlier we know whether a branch will be taken, the fewer instructions need to be flushed from the pipeline!

1. Calculate branch target address in ID stage by separate adder in ID stage

2. Test condition in ID stage using an additional comparator

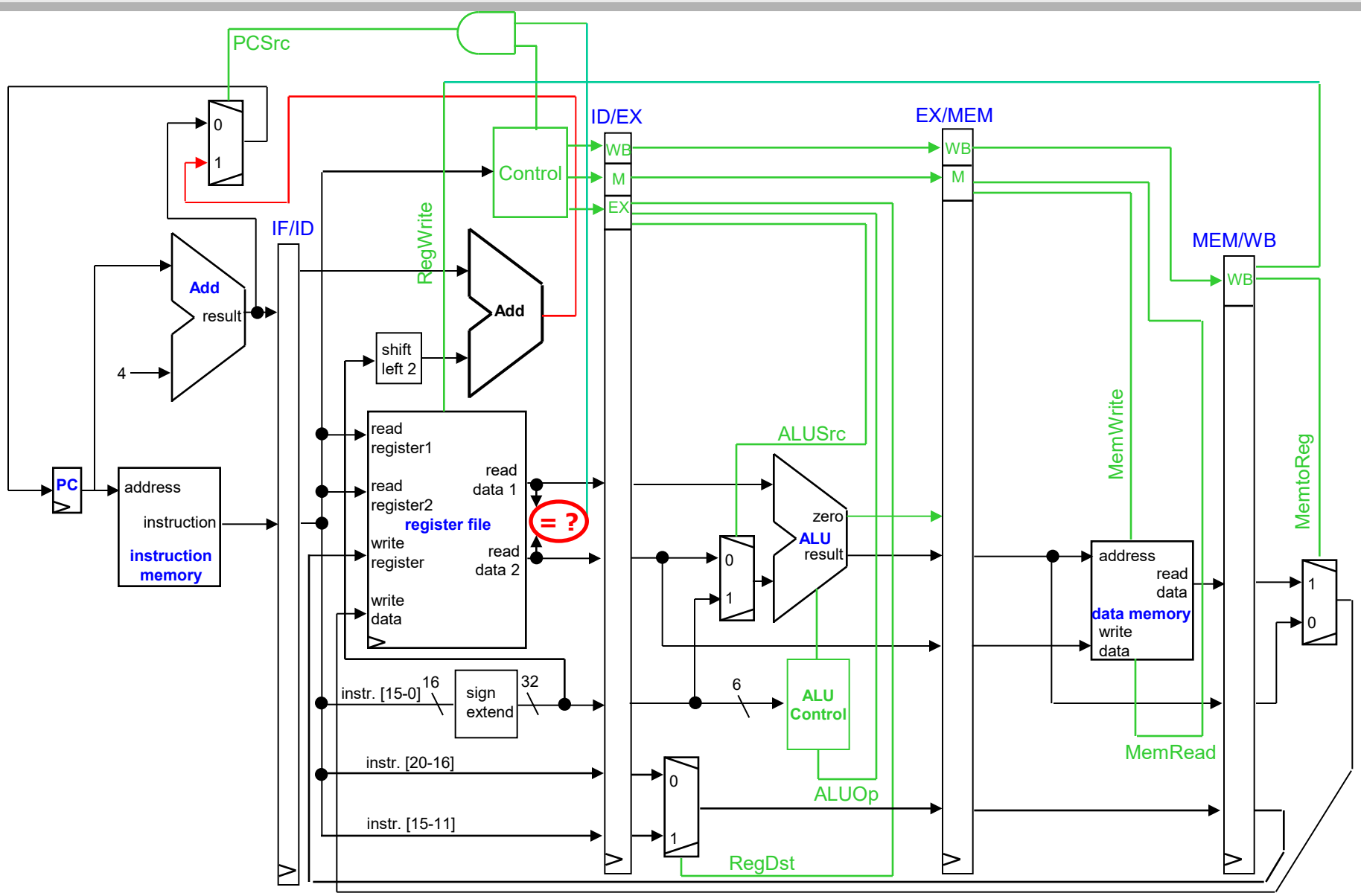
   Comparator is faster than ALU so that it can be integrated into ID Phase!

⇒ Only one instruction needs to be flushed



8 bit comparator

Reducing the delay of branches



Delayed Branches

Since we know which instruction to fetch next with a delay of only one clock cycle, we can afford "delayed branching".

Delayed branching:

No instructions are flushed from the pipeline. An instruction following immediately after a branch is always executed.

Programming strategy:

Place an instruction originally preceding the branch and not affected by it immediately after the branch (=*branch delay slot*).

If no suitable instruction is found place a NOP there.

Typically the compiler/assembler will fill about 50% of all delay slots with useful instructions.

Processors with several functional units

The times required for executing two arithmetic instructions may differ significantly depending on the type of the instruction:

- Integer addition faster than floating point addition
- Addition much faster than multiplication/division

Making the cycle time long enough so that the slowest instruction can be executed in one cycle would slow down the processor dramatically!

Solution:

- Distribute the EX stage of complex operations over several clock cycles
- Use several functional units in the EX stage

$\Rightarrow$ Allows to execute several instructions in parallel!

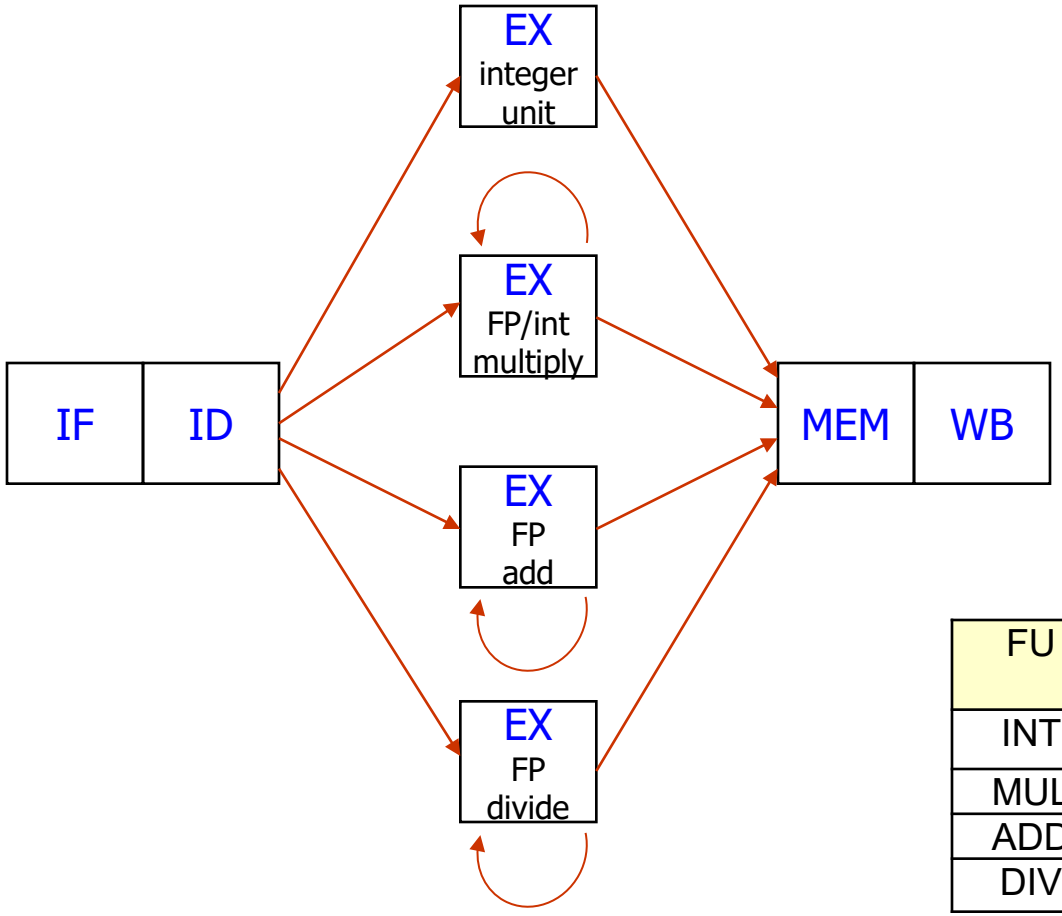
Extending the MIPS pipeline to handle multicycle floating point operations

MIPS implementation with floating point (FP) instructions (MIPS R4000):

- 1 Integer unit: used for load/store, integer ALU operations and branches
- 1 Multiplier for integer and FP numbers
- 1 Adder for FP addition and subtraction
- 1 Divider for FP and integer numbers

Extended MIPS pipeline

MIPS pipeline with multiple functional units (FUs)

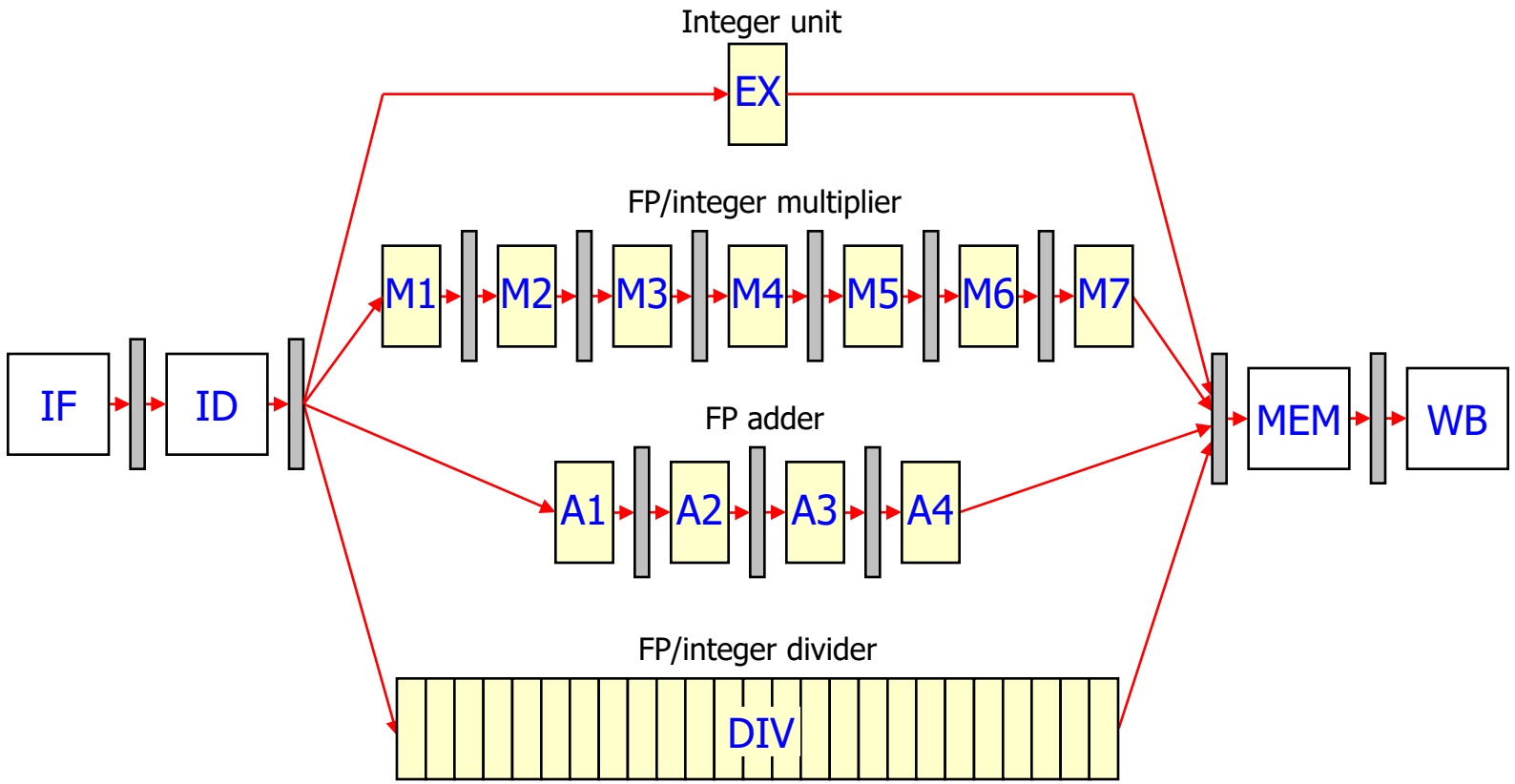


FU	Execution time	Structure
INT	1	Not pipelined
MUL	7	Pipelined
ADD	4	Pipelined
DIV	25	Not pipelined

Out of order completion possible!

Extended MIPS pipeline

MIPS pipeline with multiple functional units (FUs)



Extended MIPS pipeline

Separate register file for storing FP operands:

- FP registers f0 – f31
- FP instructions operate on FP registers
- Integer instructions operate on integer registers
- Exception: FP load/store: address in integer register, data in FP register

  + no increase in number of bits needed for addressing registers
  + simplifies hazard detection
  + read/write integer and FP operands at the same time
  + no increase in complexity of multiplexers/decoders (speed!)

  - Additional moves for copying data from FP registers to integer register and vice versa necessary

- FP operands may be 32 or 64 bit wide

  One 64 bit operand occupies a pair of FP registers (e.g. f0 and f1)

  64 bit path from/to memory to speed up double precision load/store

Structural Hazard: functional unit

Example: Floating point operations

```
Div.d $f0,$f2,$f4
Mul.d $f4,$f6,$f4
Div.d $f8,$f8,$f14
Add.d $f10,$f4,$f8
```

The `*.d` extension indicates 64 bit floating point operations

Cycle	1	2	3	4	5	6	7	8	9	10	11	12
`Div.d $f0,$f2,$f4`	IF	ID	DIV----------									
`Mul.d $f4,$f6,$f4`		IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
`Div.d $f8,$f8,$f14`			IF	ID	**stall...**							
`Add.d $f10,$f4,$f8`				IF	**stall...**							

Functional units which are not pipelined and which require more than one clock cycle for execution may create structural hazards!

Division unit causes structural hazard.

Instruction has to be stalled in ID-stage!

Structural Hazard

Example:

Cycle	1	2	3	4	5	6	7	8	9	10	11
`Mul.d $f0,$f4,$f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
`Add $r0,$r2,$r3`		IF	ID	EX	MEM	WB					
`Add $r3,$r0,$r0`			IF	ID	EX	MEM	WB				
`Add.d $f2,$f4,$f6`				IF	ID	A1	A2	A3	A4	MEM	WB
`Sw $r3,0($r2)`					IF	ID	EX	MEM	WB		
`Sw $r0,4($r2)`						IF	ID	EX	MEM	WB	
`L.d $f2,0($r2)`							IF	ID	EX	MEM	WB

Structural Hazard:

3 instructions wish to write their results to FP register file!

Solution: Track use of the write port of register file in ID stage by using a shift register. If a structural hazard would occur the instruction being in ID stage is *stalled for one cycle*

Structural Hazard: write back

Example: resolved structural hazard

Cycle	1	2	3	4	5	6	7	8	9	10	11	12	
`Mul.d $f0,$f4,$f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB		
`Add $r0,$r2,$r3`		IF	ID	EX	MEM	WB							
`Add $r3,$r0,$r0`			IF	ID	EX	MEM	WB						
`Add.d $f2,$f4,$f6`				IF	ID	**stall**	A1	A2	A3	A4	MEM	WB	
`Sw $r3,0($r2)`					IF	stall	ID	EX	MEM	WB			
`Sw $r0,4($r2)`							IF	ID	EX	MEM	WB		
`L.d $f2,0($r2)`								IF	ID	**stall**	EX	MEM	WB

Example:

Cycle	1	2	3	4	5	6	7	8	9	10	11	12
`Mul.d $f0,$f4,$f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB	
`Add $r0,$r2,$r3`		IF	ID	EX	MEM	WB						
`Add.d $f0,$f4,$f6`			IF	ID	A1	A2	A3	A4	MEM	WB		

WAW-Hazard:
Add.d writes f0 before Mul.d does

Out-of-order completion may lead to WAW hazard!

Solution: Stall `Add.d` instruction in ID stage

Cycle	1	2	3	4	5	6	7	8	9	10	11	12
`Mul.d $f0,$f4,$f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB	
`Add $r0,$r2,$r3`		IF	ID	EX	MEM	WB						
`Add.d $f0,$f4,$f6`			IF	ID	**stall**	**stall**	A1	A2	A3	A4	MEM	WB

⇒ Hazard detection logic detects all hazards in ID stage and resolves them by stalling the corresponding instruction

Extended MIPS pipeline

Instruction execution:

1. Fetch
2. Decode:
   1. *Check for structural hazards:* Wait until the required FU is not busy and make sure the register write port is available when it will be needed
   2. *Check for RAW data hazards:* Wait until source registers are not listed as destination register of any instruction in M1-M6, A1 – A3, DIV or a load in EX

   Optimization : e.g.: if the division is in the final clock cycle its result may be forwarded to the requesting FU in the cycle following.

   3. *Check for WAW data hazards:* Determine if any instruction in A1 - A4, M1 - M7, DIV, has the same destination as this instruction. If so, stall instruction for the number of clock cycles being necessary

   Simplification: Since WAW hazards are rare, stall instruction until no other instruction in the pipeline has the same destination
3. Execute
4. Memory Access
5. Write Back

Dynamic Branch Prediction

Assume branch not taken is a crude form of branch prediction. Typically it fails in 50% of all cases.

In processors with multiple functional units deep pipelines are used. This may lead to large branch delays if a branch is predicted the wrong way!

⇒ we need more accurate methods for predicting branches!

Idea: *dynamic branch prediction*
predict branches using the program's past behaviour.

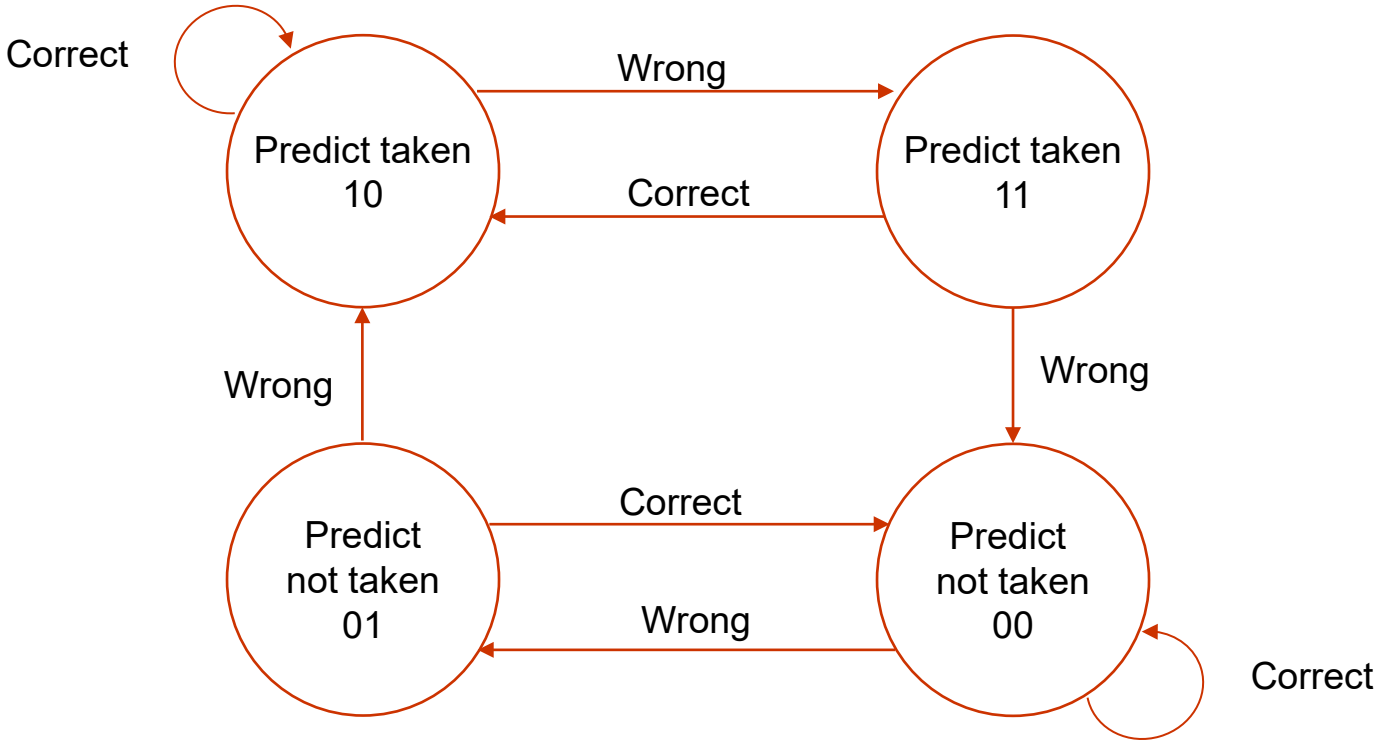
Branch target buffer and branch history table

Small memory addressed by the lower bits of the instruction address, contains a flag indicating whether the branch has been taken or not.

This flag is set or reset at each branch.

Dynamic Branch Prediction

For loops the hit rate may be improved by using two bits for branch prediction. A prediction must be wrong twice before it is changed.



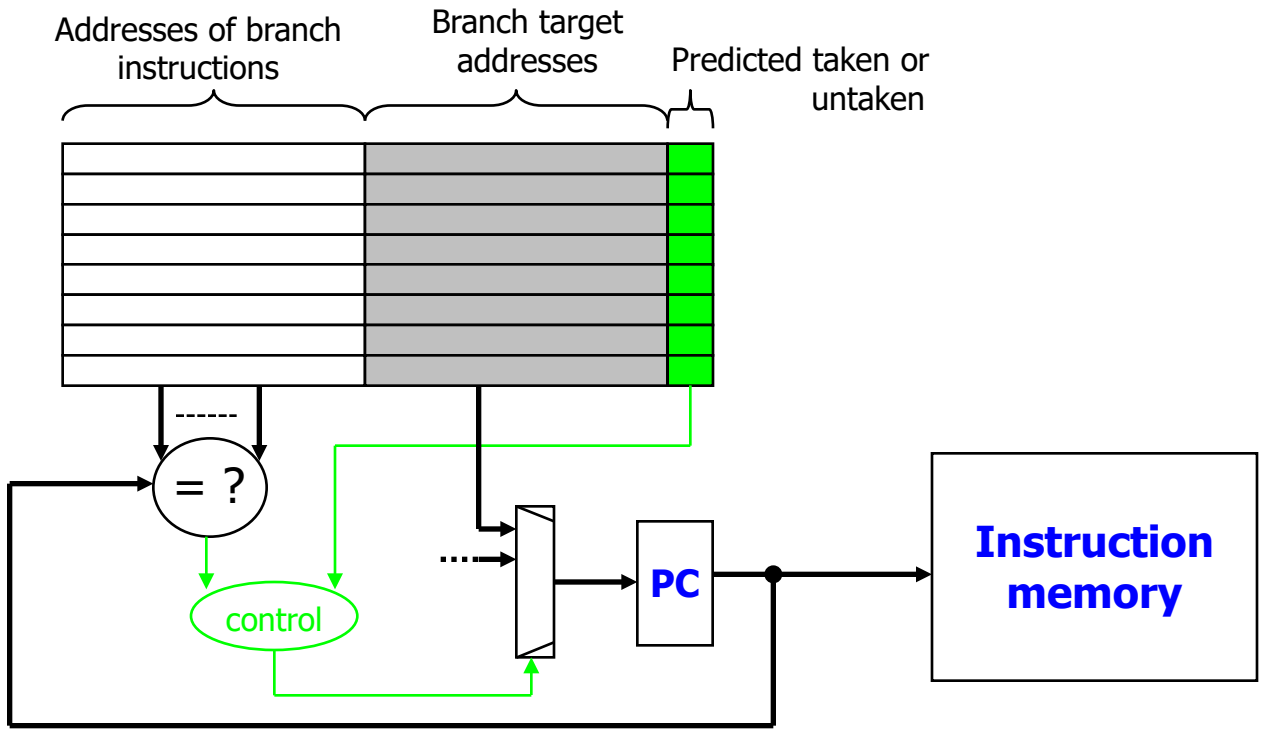
2 bit prediction scheme

More sophisticated prediction schemes include a branch history table to record previous branch behavior

Branch Target Buffer

Observation: the target address (calculated from PC and offset) for a particular branch remains constant during program execution.

Idea: store the branch target addresses in a lookup table: *branch target buffer*



Branch target buffer in combination with correct branch prediction allows to execute branches without stalling the pipeline!

Statically Scheduled Pipelines

So far: *static scheduling*

Execution is started in the order in which the instructions have been fetched. (e.g., in the order which the compiler has determined).

If a data dependence occurs that can not be resolved by forwarding, the pipeline is *stalled* (starting with the instruction that waits for a result). No new instructions are fetched until the dependence is cleared.

Dynamically Scheduled Pipelines

Idea: Hardware rearranges instruction executions dynamically to reduce stalls
$\Rightarrow$ *Dynamic Scheduling*

Dynamic Scheduling takes structural hazards and data hazards into consideration!

To avoid that an instruction is stalled because a data hazard delays all subsequent instructions, the ID stage is spilt into two stages:

1. Issue: Decode instructions, check for structural hazards
2. Read operands: Wait until no data hazards, then read operands

Leads to out-of-order execution and out-of-order completion

Out-of-order execution

Out-of-order execution may lead to WAR hazards!

Example: Floating point operations

```
Div.d $f0, $f2, $f4
Add.d $f10,$f0, $f8
Mul.d $f8, $f8, $f14
```

`Add.d` needs to be stalled because of RAW hazard.

`Mul.d` may be started,

BUT: if `mul.d` completes before `add.d` reads its operands, `add.d` will read the wrong value in `f8`!

The control logic deciding when an instruction is executed has to detect and resolve hazards!

Score Board

Dynamic Scheduling with a Score Board

Goal: maintain an execution rate of one instruction per clock cycle by executing an instruction as early as possible

If an instruction needs to be stalled because of a data hazard other instructions can be issued and executed.

⇒ We have to analyze the program flow for hazards!

Scoreboard:

- Detects structural hazards and data hazards
- Determines when an instruction may read operands and when it is executed
- Determines when an instruction can write its result into the destination register

Dynamic Scheduling with a Score Board

In the following we will consider dynamic scheduling only for arithmetic instructions – no MEM access-phase necessary.

4 stages (replace ID, EX, WB stage of standard MIPS pipeline):

1. Issue: If

   - a functional unit (FU) for the instruction is free (resolve structural hazards)

   and

   - no other active instruction has the same destination register (resolve WAW hazards)

   the score board issues the instruction to the FU and updates its internal data structure.

   If a hazard exists, the issue stage stalls. Subsequent instructions are written into a buffer between instruction fetch and issue. If this buffer is filled then the instruction fetch stage stalls.

Dynamic Scheduling with a Score Board

2. Read operands: When all operands are available, the score board tells the FU to read its operands and to begin execution (may lead to out of order execution).

   A source operand is available, when no active instruction issued earlier is going to write it (resolve RAW hazards).

3. Execution: The FU executes the instruction (may take several clock cycles). When the result is ready, the FU notifies the scoreboard that it has completed execution.

Dynamic Scheduling with a Score Board

4. Write result: When an FU announces the completion of an execution, the scoreboard checks for WAR hazards. If no such hazard exists, the result can be written to the destination register. A WAR hazard occurs when there is an instruction preceding the completing instruction that

- has not read its operands yet and
- one of these operands is the same register as the destination register of the completing instruction.

Score Boarding does not use forwarding!
If no WAR hazard occurs, the result is written to the destination register during the clock cycle following the execution. (We do not have to wait for a statically assigned WB stage that may be several cycles away).

Example

MIPS processor with dynamic scheduling using a score board with the following functional units (not pipelined) in the datapath:

- 1 Integer unit: for load/store, integer ALU operations and branches
- 2 Multipliers for FP numbers
- 1 Adder for FP addition/subtraction
- 1 Divider FP numbers

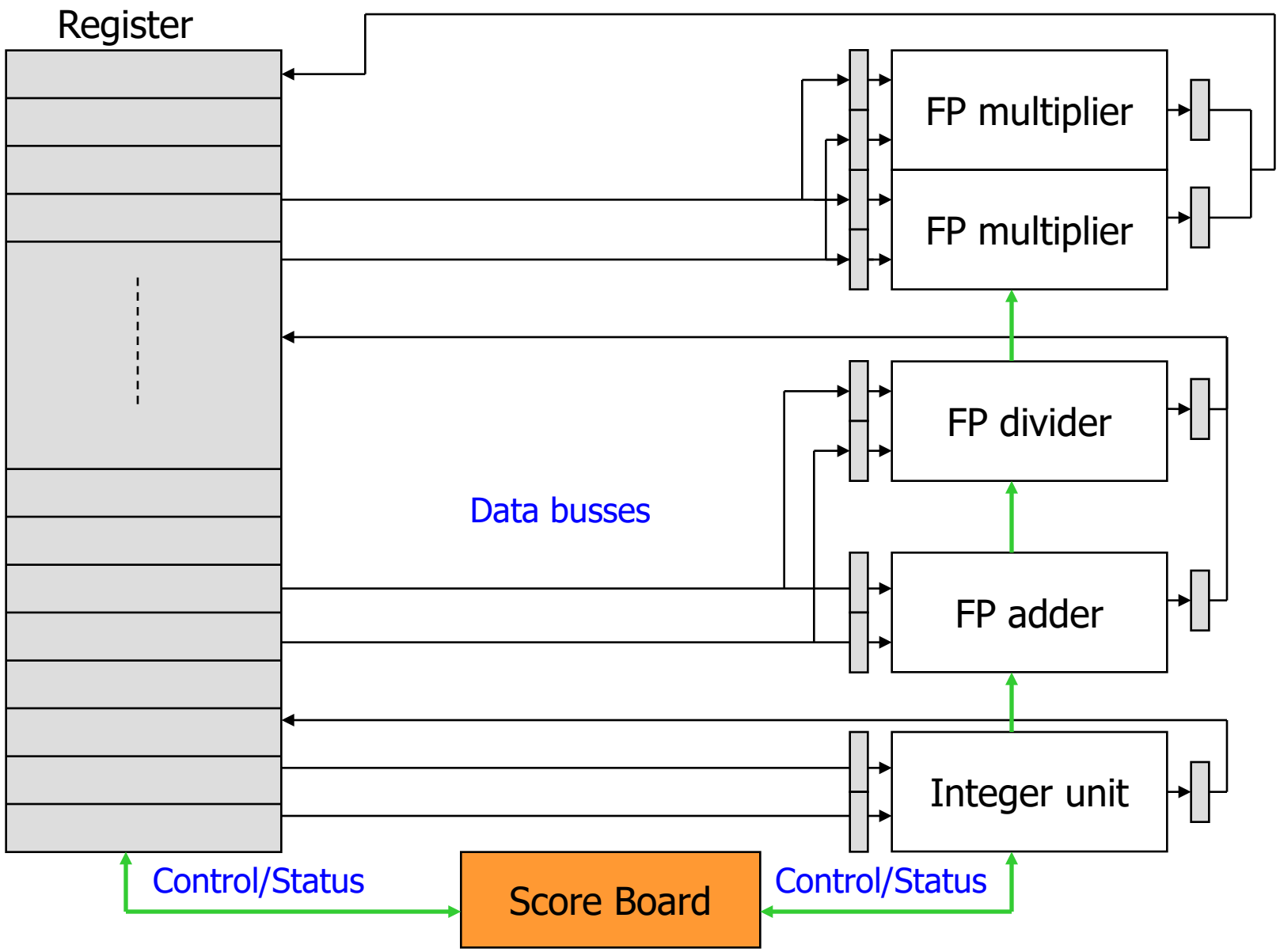
MIPS program with floating point instructions (64 Bit):

```
L.d $f6, 34 ($r2)
L.d $f2, 45 ($r3)
Mul.d $f0, $f2, $f4
Sub.d $f8, $f2, $f6
Div.d $f10, $f0, $f6
Add.d $f6, $f8, $f2
```

Assumptions:
EX phase for double precision takes:

2 cycles for load and add (pipelined)
10 cycles for mult (pipelined)
40 cycles for div (not pipelined)

MIPS with a Score Board



Components of the Score Board

Score Board consists of three parts containing the following data:

1. Instruction status: indicates for each instruction which of the four steps the instruction is in

2. FU status: indicates for each FU its state:

   busy: FU busy or not
   OP: Operation to perform (e.g. add or subtract)
   f_i: Destination register
   f_j, f_k: Source registers
   Q_j, Q_k: Functional units writing the source registers f_j und f_k
   R_j, R_k: Flags indicating whether f_j and f_k are ready to be read but have not been read yet. Are set to "no" after the operands have been read.

3. Result register status: indicates for each register, whether a FU is going to write it and which FU this will be

Components of the Score Boards

Instruction	Issue	Read operands	Execution complete	Write result
`L.d $f6, 34 ($r2)`	√	√	√	√
`L.d $f2, 45 ($r3)`	√	√	√	
`Mul.d $f0, $f2, $f4`	√			
`Sub.d $f8, $f2, $f6`	√			
`Div.d $f10, $f0, $f6`	√			
`Add.d $f6, $f8, $f2`				

Instruction status

Name	Busy	Op	f_i	f_j	f_k	Q_j	Q_k	R_j	R_k
integer	yes	load	f2	r3		0		no	
mult1	yes	mult	f0	f2	f4	integer	0	no	yes
mult2	no								
add	yes	sub	f8	f2	f6	integer	0	no	yes
divide	yes	div	f10	f0	f6	mult1	0	no	yes

Functional unit status

(double precision floating point numbers number ⇒ allocate two 32 bit registers)

	f0	f2	f4	f6	f8	f10	f12	…	f30
FU	mult1	integer	0	0	add	divide	0		0

Result register status

Bookkeeping in the Score Board

When an instruction has passed through one step the score board is updated.

FU:	FU used by instruction	f_i[FU], f_j[FU], f_k[FU]:	destination/source registers of FU
d:	destination register	R_j[FU], R_k[FU]:	s_1, s_2 ready?
s_1, s_2:	source registers	Q_j[FU], Q_k[FU]:	FUs producing s_1 and s_2
op:	type of operation	Result[d]:	FU that will write register d
		Op[FU]:	operation which FU will execute

Instruction status	Wait until	Bookkeeping
Issue	Busy[FU] = no and Result[d] = 0 (no other FU has d as destination register)	Busy[FU] := yes; Op[FU] := op; Result[d] := FU; f_i[FU] := d; f_j[FU] := s1; f_k[FU] := s2; Q_j := Result[s1]; Q_k := Result[s2]; if Q_j = 0 then R_j := yes; else R_j := no if Q_k = 0 then R_k := yes; else R_k := no
Read operands	R_j = yes and R_k = yes	R_j := no; R_k := no; Q_j := 0; Q_k := 0
Execution	Functional unit done	
Write results	$\forall f((f_j[f] \neq f_i[\text{FU}]$ or $R_j[f]$ = no) and $(f_k[f] \neq f_i[\text{FU}]$ or $R_k[f]$ = no))	$\forall f$(if $Q_j[f]$ = FU then $R_j[f]$:= yes); $\forall f$(if $Q_k[f]$ = FU then $R_k[f]$:= yes); Result[f_i[FU]] := 0; Busy[FU] := no

for all FUs

Bookkeeping in the Score Board

Comment for step write results:

$\forall f (f_j[f] \neq f_i[FU]$ or $R_j[f] = no)$

„$R_j[f] = no$" means, that the instruction which is now active at *f* will not read the current contents of source register f_j

a) either, since the operation has already been executed and currently waits for permission to write, or

b) since the required source operand must still be computed and the current instruction is waiting for that.

In the first case register f_j is overwritten since the previous contents are no longer needed. In the second case the register is overwritten since this will provide the expected operand.

$R_i[f] = yes$ means that the instruction being active at *f* still requires the current content of the register specified by f_i.

Dynamic Scheduling: Tomasulo's Schema

Are there further possibilities for eliminating stalls resulting from hazards?

RAW hazard: No way - we have to wait until all operands are calculated!

WAR hazard and WAW hazard:

Example:

```
div.d $f0, $f2,  $f4
add.d $f6, $f0,  $f8     RAW hazard for f0
sub.d $f8, $f10, $f14    WAR hazard for f8
mul.d $f6, $f10, $f8     WAW hazard for f6, RAW hazard for f8
```

Dynamic Scheduling: Tomasulo's Schema

Are there further possibilities for eliminating stalls resulting from hazards?

RAW hazard: No way - we have to wait until all operands are calculated!

WAR hazard and WAW hazard:

Example:

```
div.d $f0, $f2,  $f4
add.d $f6, $f0,  $f8     RAW hazard for f0
sub.d $f8, $f10, $f14    WAR hazard for f8
mul.d $f6, $f10, $f8     WAW hazard for f6, RAW hazard for f8
```

Observation: WAR and WAW hazard could have been avoided by compiler!

Idea: *Register renaming*

Rename destination registers of instructions in a way that prevents instructions being executed *out-of-order* from overwriting operands being still required by other instructions ⇒ Tomasulo's scheme or Tomasulo's algorithm

Register renaming

Example (continued):

Assume we have two temporary registers `S` and `T`.

replace `f6` in `add.d` by a temporary register `S` and
replace `f8` in `sub.d` and `mul.d` by a temporary register `T`:

```
div.d $f0, $f2, $f4
add.d $S, $f0, $f8
sub.d $T, $f10, $f14
mul.d $f6, $f10, $T
```

Replace target registers affected by a WAW or a WAR hazard by temporary registers and modify subsequent instructions reading these registers appropriately.

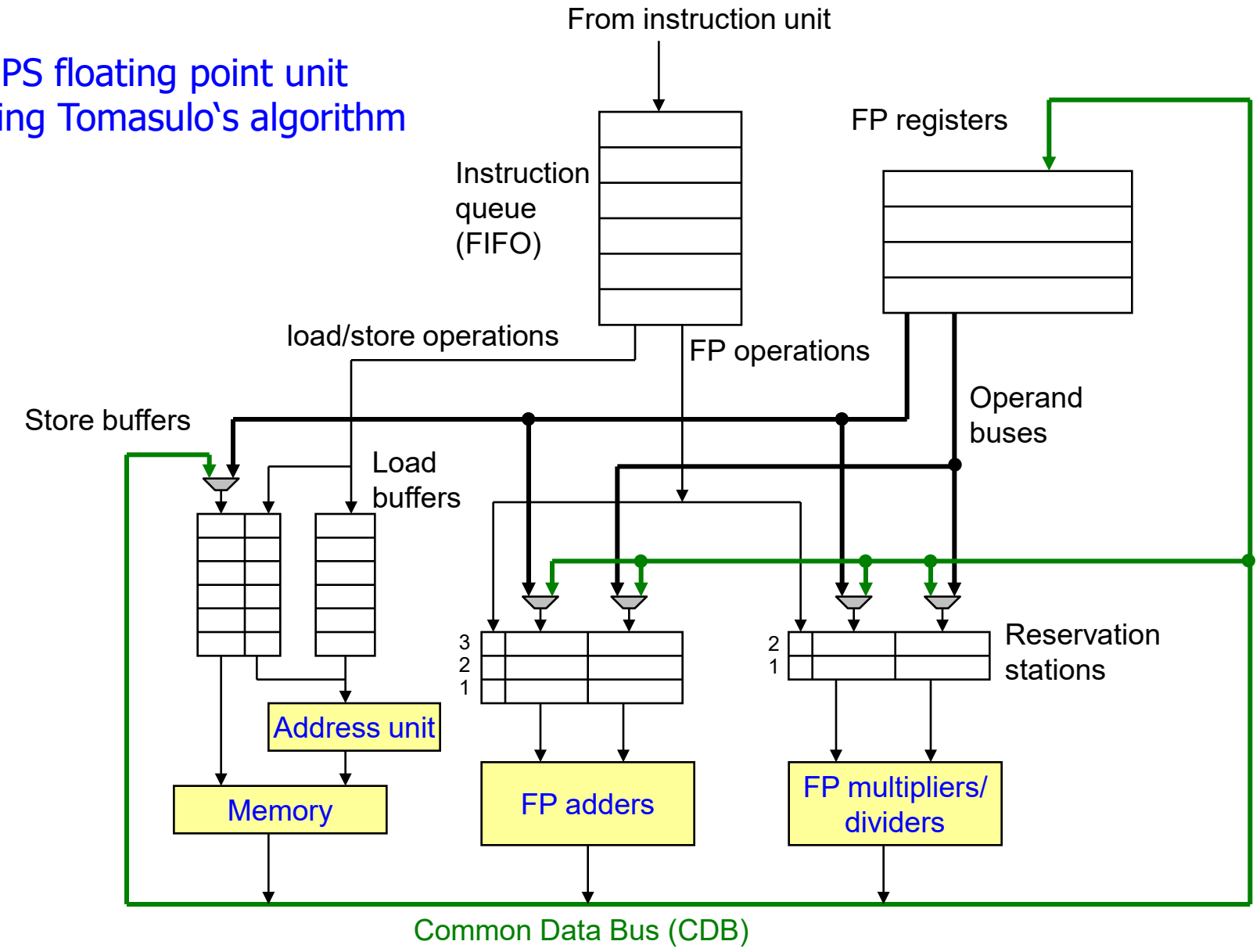
Reservation Station

Temporary registers are part of *reservation stations* :

- Buffer the operands for instructions waiting for execution.
  If an operand is not yet calculated, the corresponding reservation station contains the number of the reservation station which will deliver the result.
- *Renaming* of register numbers for pending operands to the names of the reservation stations. This is done during instruction issue.
- Information about the availability of the operands stored in a reservation station determines when the corresponding instruction can be executed.
- As results become available, they are sent directly from the reservation stations to the waiting FU over the *common data bus* (*CDB*)
- When successive writes to a register overlap in execution, only the result of the instruction being issued at last is used to update the register.
  $\Rightarrow$ resolves WAR/WAW hazards

Tomasulo's algorithm

MIPS floating point unit
using Tomasulo's algorithm



Tomasulo's algorithm - stages

Load/store buffers storing data/addresses coming from and going to memory behave similarly like reservation stations for arithmetic units.

Steps in execution of a FP instruction:

1. Issue:

Get next instruction from the head of the instruction queue and issue it to a matching reservation station that is empty.

Operands available in registers?

yes: hand over values to reservation station

no: hand over names of those reservation stations that are calculating the values

Buffering operands resolves WAR hazards!

If no matching *reservation station* is empty, there is a structural hazard.

$\Rightarrow$ instruction stalls until a station is freed

Tomasulo's algorithm - stages

2. Execution

1. If one or more of the operands are not available, monitor the CDB.
2. If an operand becomes available, place it in the waiting reservation station(s).
3. Wait until all operands for an instruction are available, then start execution ⇒ resolves RAW hazards

In case of stores: Execution may start (address calculation) even if data to be stored is not available yet. Address calculation unit is occupied during address calculation only.

3. Write result

1. When the result is available, send it to the CDB.
2. From the CDB it is sent directly to waiting reservation stations (and store buffers)

   Only if this instruction is the last one being issued that writes to a certain target register, write the result to that target register ⇒ avoids WAW hazards

Reservation stations

Each *reservation station* has the following fields:

Op: Type of the operation to perform (e.g. add or subtract)

Q_j, Q_k: Names of the *reservation stations* containing the instructions calculating the operands. Zero values indicate that the operands are already available

V_j, V_k: Values of the source operands

Busy: Flag indicating that this station/buffer is already occupied.

Each *load/store buffer* has an additional field:

A: Initially the immediate field of the address is stored there; after address calculation the effective address is stored there

For each register of the register file there is one field

Q_i: Name of the *reservation station* containing the instruction being issued last that calculates the result for this register. A zero value indicates that no active instruction is calculating a result for that register.

Tomasulo's method: information tables

Instruction	Issue	execute	Write result
`L.d $f6, 34 ($r2)`	√	√	√
`L.d $f2, 45 ($r3)`	√	√	
`Mul.d $f0, $f2, $f4`	√		
`Sub.d $f8, $f2, $f6`	√		
`Div.d $f10, $f0, $f6`	√		
`Add.d $f6, $f8, $f2`	√		

Instruction status

Name	Busy	Op	V_j	V_k	Q_j	Q_k	A
load1	no						
load2	yes	load					45+Regs[r3]
add1	yes	sub		Mem[34+Regs[r2]]	load2		
add2	yes	add			add1	load2	
add3	no						
mult1	yes	mul		Regs[f4]	load2		
mult2	yes	div		Mem[34+Regs[r2]]	mult1		

Reservation Stations

Register:	f0	f2	f4	f6	f8	f10	f12	…	f30
Q_i :	mult1	load2	0	add2	add1	mult2	0		0

Register status

Dynamic Scheduling: Data hazards through memory

A load and a store instruction may only be done in a different order if they access different addresses! (**RAW/WAR hazard!**)

Two stores sharing the same data memory address may not be done in different order! (**WAW hazard!**)

Load: read memory only if there is no uncompleted store which has been issued earlier and which shares the same data memory address with the load.

Store: write data only if there are no uncompleted loads and stores being issued earlier using the same data memory address as the store.

Dynamic Scheduling: Instructions following branches

It may take many clock cycles until we know whether a branch has been predicted correctly or not!

1. Instructions being issued after a branch may complete before.

⇒ Write back stage of these instructions has to be stalled until we know whether the prediction has been correct or not!

2. Exceptions:

We have to ensure that exactly the same *exceptions* are handled as in the case where the pipeline would have used in-order-execution and no branch prediction!

Simple solution:
Instructions following a branch are issued only. Execution starts only after the branch prediction has turned out to be correct.

⇒ Can reduce the efficiency of a dynamically scheduled pipeline dramatically!

Speculative execution

Write result stage is spilt into two stages: write results + commit

3. Write results:

- Instructions are executed as operands become available. Results are written into a **reorder buffer (ROB)**.

- For each active instruction there is one entry in the ROB. Their order corresponds to the order in which the instructions have been issued.

  ⇒ The head of the ROB contains the result of the active instruction being issued first.

  Subsequent instructions can read their operands from the ROB.

- Writes going to register file and memory are delayed until branch predictions turn out to be correct.

Speculative execution

4. Commit:

- When an instruction that writes to memory or register file reaches the head of the ROB, its result is written. *Exception is handled now if necessary!*

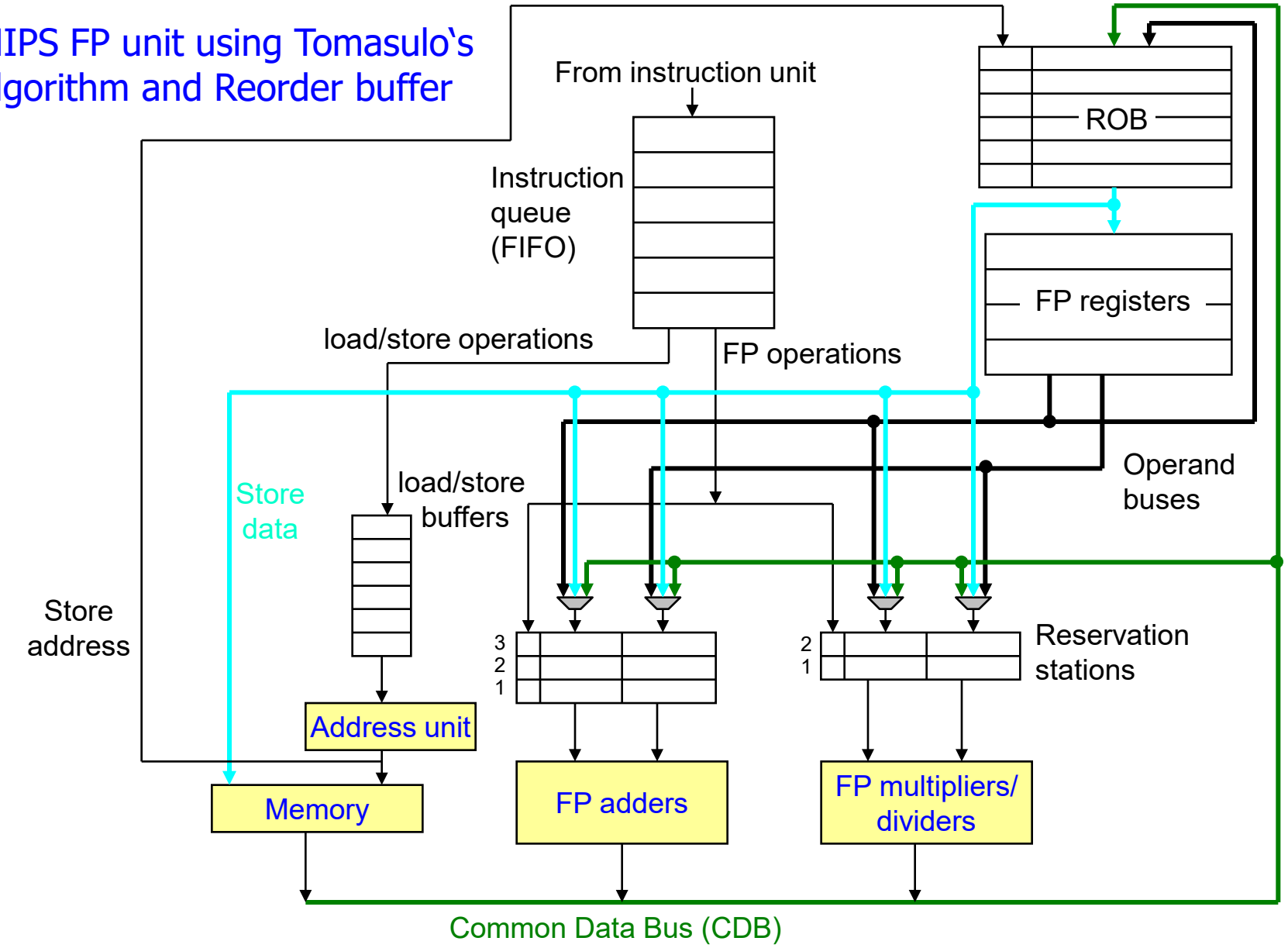
- If the *head* of the ROB contains an incorrectly predicted branch, the ROB is flushed.

  ⇒ results calculated by instructions following the branch are discarded!

ROB restores initial order of instructions: in-order-commitment

Speculative Execution

MIPS FP unit using Tomasulo's algorithm and Reorder buffer



Multiple Issue Processor

Using *multiple FUs*, *dynamic scheduling*, *branch prediction* and *speculation* allow to achieve an CPI of nearly one.

CPI < 1 not possible because we issue only one instruction per clock cycle!

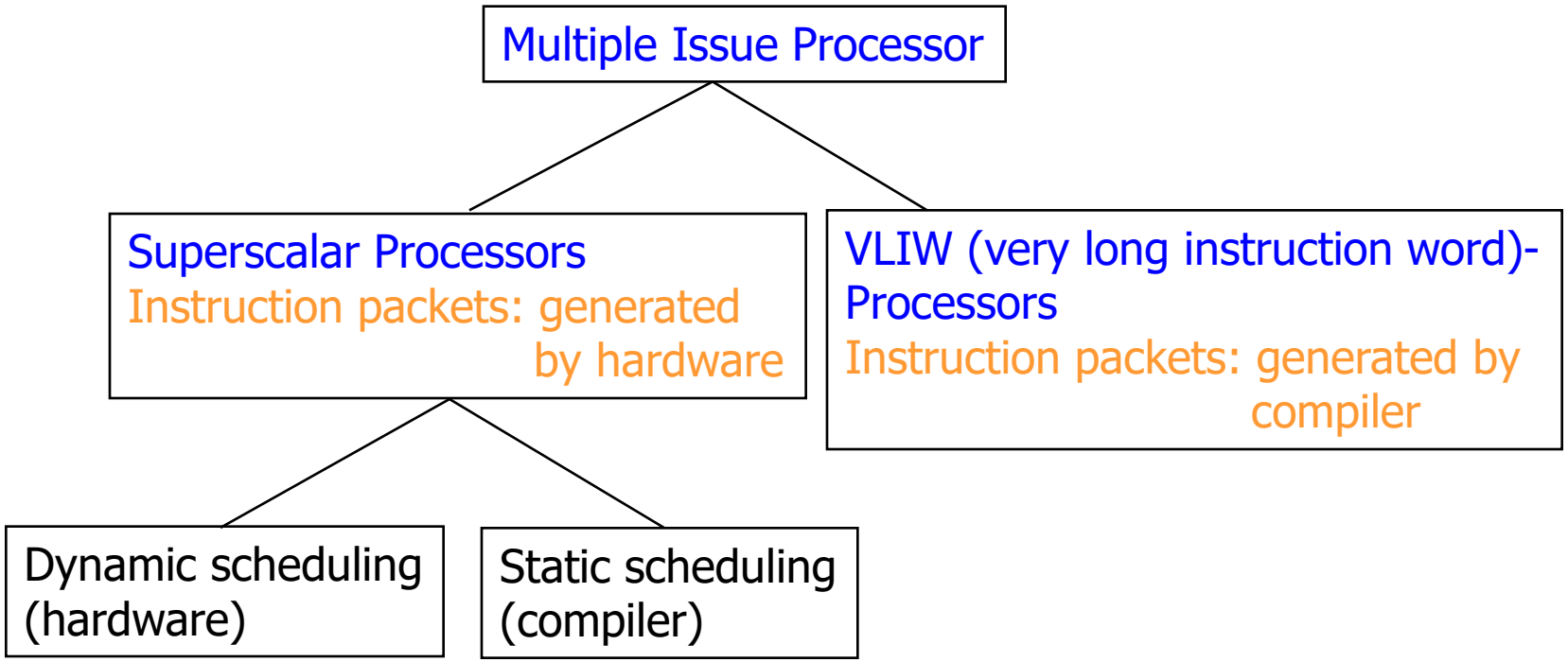
Further speedup:

Issue multiple instructions in one clock cycle (up to 8 in practice)

$\Rightarrow$ CPI < 1 possible!

The sets of instructions being issued in parallel are called instruction packets or issue packets.

Multiple Issue Processors



Overview

Name	Issue	Hazard Detection	Scheduling	Distinguishing characteristics	Examples
superscalar (static)	dynamic	Hardware	static (Compiler)	in-order execution	Sun UltraSPARC II/III ARM Cortex-A8
superscalar (dynamic)	dynamic	Hardware	dynamic	out-of-order execution	IBM Power PC
superscalar (speculative)	dynamic	Hardware	dynamic with speculation	out-of-order execution with speculation	Pentium III/4, MIPS R10K, Alpha 21264, HP PA 8500, IBM RS64III ARM Cortex-A9 RISC-V BOOM
VLIW	static	software (Compiler)	static (Compiler)	no hazards between issue packets	Trimedia, i860 AMD R600-RV870

Statically scheduled superscalar Processors

Example: dual-issue static superscalar processor

In one clock cycle we can issue

- one integer instruction (including load/store, branches, integer ALU operations) and
- one arithmetic FP instruction

Only slight extensions of the hardware necessary compared to a single issue implementation with two FUs.

Typical for high-end embedded processors.

Statically scheduled Dual Issue Pipeline

Instruction type	Pipeline stages								
Integer instruction	IF	ID	EX	MEM	WB				
FP instruction	IF	ID	EX	EX	EX	WB			
Integer instruction		IF	ID	EX	MEM	WB			
FP instruction		IF	ID	EX	EX	EX	WB		
Integer instruction			IF	ID	EX	MEM	WB		
FP instruction			IF	ID	EX	EX	EX	WB	
Integer instruction				IF	ID	EX	MEM	WB	
FP instruction				IF	ID	EX	EX	EX	WB

CPI of 0.5 possible !

Multiple Issue Pipeline

In order to enable multiple issues per clock cycle we must be able to fetch multiple instructions per cycle also!

Example: 4-way issuing processor

Fetches instructions stored at PC, PC+4, PC+8, PC+12 from memory

⇒ Wide bus to instruction memory required!

Problem: what if one of these instruction is a branch?

1. Reading branch target buffer and accessing instruction memory in one clock cycle would increase cycle time.

2. If *n* instructions of the package are allowed to be a branch, we would have to lookup *n* instructions in the branch target buffer in parallel!

Typical simplification: single issue for branches

Multiple issue with dynamic pipelining (Tomasulo)

Example: Superscalar processor with:

- Dual issue (single issue for branches)
- Dynamic Tomasulo scheduling (no speculation, i.e. execution of instructions following branch must be delayed until the branch condition is evaluated)
- One FP unit
- One FU for integer instructions and load/stores and branch condition testing
- Separate FU for branch address calculation
- Several reservation stations/load store buffers for each FU: Load/stores occupy the FU only during address calculation, branches only during condition testing; Stores are allowed to execute even if the data to be stored is not available yet.

```
Loop: l.d   $f0, 0 ($r1);  # f0 := array element
      add.d $f4, $f0, $f2; # add f2 to f0
      s.d   $f4, 0 ($r1);  # store result
      addi  $r1, $r1, -8;  # decrement pointer
      bne   $r1, $r2, LOOP;# repeat loop if r1 ≠ r2
```

Latency: Number of cycles from the beginning of execution step to the moment when the result is available on the CDB

Integer operations:	1 cycle
Load:	2 cycles (1 in EX stage + 1 in MEM stage)
FP operation:	3 cycles (in EX stage)

Multiple issue with dynamic pipelining

Iteration number	Instruction	Issues at	Executes at	Memory access at	Write CDB at	Comment
1	`L.d $f0, 0($r1)`	1	2	3	4	First issue
1	`Add.d $f4, $f0, $f2`	1	5 - 7		8	Wait for l.d
1	`S.d $f4, 0 ($r1)`	2	3	9		Wait for add.d
1	`addi $r1, $r1, -8`	2	4		5	Wait for ALU
1	`bne $r1, $r2, LOOP`	3	6			Wait for addi
2	`L.d $f0, 0($r1)`	4	7	8	9	Wait for bne
2	`Add.d $f4, $f0, $f2`	4	10 - 12		13	Wait for l.d
2	`S.d $f4, 0 ($r1)`	5	8	14		Wait for add.d
2	`addi $r1, $r1, -8`	5	9		10	Wait for ALU
2	`bne $r1, $r2, LOOP`	6	11			Wait for addi
3	`L.d $f0, 0($r1)`	7	12	13	14	Wait for bne
3	`Add.d $f4, $f0, $f2`	7	15 - 17		18	Wait for l.d
3	`S.d $f4, 0 ($r1)`	8	13	19		Wait for add.d
3	`addi $r1, $r1, -8`	8	14		15	Wait for ALU
3	`bne $r1, $r2, LOOP`	9	16			Wait for addi

Resource usage

Clock cycle	Integer unit	FP unit	Data memory	CDB
2	1 / l.d			
3	1 / s.d		1 / l.d	
4	1 / addi			1 / l.d
5		1 / add.d		1/ addi
6	1 / bne	1 / add.d		
7	2 / l.d	1 / add.d		
8	2 / s.d		2 / l.d	1 /add.d
9	2 / addi		1 / s.d	2/ l.d
10		2 / add.d		2 / addi
11	2 / bne	2 / add.d		
12	3 / l.d	2 / add.d		
13	3 / s.d		3 / l.d	2 / add.d
14	3 / addi		2 / s.d	3/ l.d
15		3 / add.d		3 / addi
16	3 / bne	3 / add.d		
17		3 / add.d		
18				3 / add.d
19			3 / s.d	
20				

Example

CPI significantly greater than 0.5:

Problem: Integer unit used for memory address calculation, for incrementing pointer and for condition test
$\Rightarrow$ branch execution is delayed by one cycle

Possible solution: additional integer FU

Problem: The execution step of an instruction following a branch has to be delayed until the branch is executed

Possible solution: use *speculative execution*

Example: Dual-issue processor with speculative execution

In order to achieve a CPI < 1 we must allow two instructions to commit in parallel!

$\Rightarrow$ More buses required

Compiler techniques

Observation: If branch prediction is perfect then loops are unrolled automatically by the hardware. Operations that belong to different iterations of the loop overlap.

Loops may be unrolled in advance by the compiler also!

⇒ Improves performance for processors without speculative execution

```
Loop: lw $t0, 0 ($s1);
      add $t0, $t0, $s2;
      sw $t0, 0 ($s1);
      addi $s1, $s1, -4;
      bne $s1, $zero, LOOP;
```

Loop before unrolling

Register renaming done by compiler!

```
Loop: addi $s1, $s1, -16;
      lw $t0, 16($s1);
      add $t0, $t0, $s2;
      sw $t0, 16($s1);
      lw $t1, 12($s1);
      add $t1, $t1, $s2;
      sw $t1, 12($s1);
      lw $t2, 8($s1);
      add $t2, $t2, $s2;
      sw $t2, 8($s1);
      lw $t3, 4($s1);
      add $t3, $t3, $s2;
      sw $t3, 4($s1);
      bne $s1, $zero, LOOP;
```

Loop after unrolling

Summary

Superscalar processors determine during program execution how many instructions are issued in one clock cycle.

Statically scheduled:

- Must detect dependences in instruction packets and resolve them by inserting stalls
- Needs assistance of the compiler for achieving a high amount of parallelism.
- Simple hardware

Dynamically scheduled:

- Requires less assistance of the compiler
- Hardware is much more complex

Static Multiple Issue – VLIW (DSP) approach

For highly superscalar processors the hardware becomes very complex.

Idea: let the compiler do as much work as possible!

VLIW approach: used for digital signal processing (DSP)

compiler groups instructions with no dependences between that may be executed in parallel into a „very long instruction word" (**VLIW**).

⇒ no hardware for hazard detection and scheduling necessary

Does the program contain enough parallelism?

The compiler has to find enough parallelism for using the full capacity of all functional units!

local scheduling : scheduling inside lists of instructions without branches (= basic blocks)

global scheduling : scheduling over several basic blocks

Example

For VLIW processors one instruction must contain all operations that are executed in parallel explicitly. Therefore, VLIW processors are sometimes also called **EPIC**s (explicitly parallel instruction computer).

Example

```
Loop: lw.d   $f0, 0($r1);
      add.d $f4, $f0, $f2;
      sw.d   $f4, 0($r1);
      addi  $r1, $r1, -8;
      bne   $r1, $r2, LOOP;
```

unroll →

```
Loop: lw.d $f0, 0($r1);
      add.d $f4, $f0, $f2;
      sw.d $f4, 0($r1);
      lw.d $f6, -8($r1);
      add.d $f8, $f6, $f2;
      sw.d $f8, -8($r1);
      lw.d $f10, -16($r1);
      add.d $f12, $f10, $f2;
      sw.d $f12, -16($r1);
      lw.d $f14, -24($r1);
      add.d $f16, $f14, $f2;
      sw.d $f16, -24($r1);
      …
      addi $r1, $r1, -56;
      bne $r1, $r2, LOOP;
```

Consider a VLIW processor with:

- 2 FUs for memory access (2 cycles)
- 2 FUs for FP operations (Pipelined, 3 cycles for EX)
- 1 FU for integer operations and branches (1 cycle)

Create a schedule for 7 iterations using loop unrolling.

Static Multiple Issue – VLIW approach

Memory unit 1	Memory unit 2	FP unit 1	FP unit 2	Integer unit
lw.d $f0,0($r1)	lw.d $f6,-8($r1)			
lw.d $f10,-16($r1)	lw.d $f14,-24($r1)			
lw.d $f18,-32($r1)	lw.d $f22,-40($r1)	add $f4,$f0,$f2	add $f8,$f6,$f2	
lw.d $f26,-48($r1)		add $f12,$f10,$f2	add $f16,$f14,$f2	
		add $f20,$f18,$f2	add $f24,$f22,$f2	
sw.d $f4,0($r1)	sw.d $f8,-8($r1)	add $f28,$f26,$f2		
sw.d $f12,-16($r1)	sw.d $f16,-24($r1)			addi $r1,$r1,-56
sw.d $f20,24($r1)	sw.d $f24,16($r1)			
sw.d $f28,8($r1)				bne $r1,$r2,Loop

Each row corresponds to a VLIW instruction